



Republic of Tunisia

Ministry of Higher Education  
and Scientific Research

Private Higher School of Engineering and Technology

TEK-UP



## FINAL YEAR PROJECT REPORT

Submitted in partial fulfillment of the requirements for the degree of

National Engineering Diploma in Applied Sciences and Technology

Specialization: Software Engineering and Information Systems

Prepared by

Rayen Aouechria

---

# Design and Development of an Integrated System for Real-Time EV Charging Station Monitoring and Technician Mission Management

---

|                          |                     |           |
|--------------------------|---------------------|-----------|
| Professional Supervisor: | Mr. Meher Tebourbi  | CEO       |
| Academic Supervisor:     | Mr. Mohamed Alouani | Professor |



I authorize the student to submit his internship report for the purpose of defense.

Professional Supervisor, **Mr. Wael KAOUECH**

**Signature and Stamp**

I authorize the student to submit his internship report for the purpose of defense.

Academic Supervisor, **Ms. Ahlem ARFAOUI**

**Signature**

---

# Dedication

*To my mother and father,*

the only two people who have known me since my first breath.

You poured everything you had into one child —  
your dreams, your sacrifices, your sleepless nights, your silent prayers.  
You gave me the world, and asked for nothing but my happiness.

*To my friends,*

the brothers and sisters I never had.

You filled the silence, shared the weight,  
and made sure I always had someone to call.  
You are my chosen family, my laughter, my home.

*To my parents, again — because you deserve to be said twice,*

every page of this work carries your name in invisible ink.

Every success is your victory.  
Every step forward is because you never let me walk alone.

*For those who left too soon,*

you are not here to hold this degree with me,  
but I carry you in every heartbeat and every triumph.  
This is as much yours as it is mine.

---

**This is for you.**

---

# Acknowledgments

*With sincere gratitude...*

I would like to express my deepest and most sincere gratitude to all those who, through their guidance, support, and encouragement, have contributed to the successful completion of this project.

First and foremost, I extend my heartfelt thanks to my professional supervisor,

**Mr. Meher Tebourbi**

whose invaluable guidance, constructive feedback, and unwavering support have been a constant source of inspiration throughout this internship. His expertise and dedication have greatly enriched this work.

I am equally and profoundly grateful to my academic supervisor,

**Mr. Mohamed Alouani**

for his thoughtful follow-up, constant encouragement, and wise academic counsel. His patience and belief in my abilities have been truly motivating.

My sincere appreciation also goes to the entire team at

**Entityservice**

for their warm welcome, exceptional collaboration, and continuous assistance during my internship. The friendly and stimulating environment they provided made this experience both enriching and memorable.

---

# Table of Contents

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>List of Abbreviations</b>                          | <b>x</b>  |
| <b>Introduction générale</b>                          | <b>1</b>  |
| <b>1 Project Context and Objectives</b>               | <b>3</b>  |
| 1.1 Introduction . . . . .                            | 4         |
| 1.2 Presentation of the Host Company . . . . .        | 4         |
| 1.2.1 EntityService Presentation . . . . .            | 4         |
| 1.2.2 Mission and Vision of the Company . . . . .     | 4         |
| 1.2.3 Organizational Chart . . . . .                  | 4         |
| 1.3 Project Presentation . . . . .                    | 5         |
| 1.3.1 Existing Systems Study . . . . .                | 5         |
| 1.3.2 Critique of Existing Systems . . . . .          | 6         |
| 1.3.3 Proposed Solution . . . . .                     | 6         |
| 1.4 Project Management Methodology . . . . .          | 7         |
| 1.4.1 Scrum Roles . . . . .                           | 7         |
| 1.4.2 Scrum Events . . . . .                          | 7         |
| 1.4.3 Scrum Artifacts . . . . .                       | 8         |
| 1.5 Modeling Language . . . . .                       | 9         |
| 1.6 Conclusion . . . . .                              | 10        |
| <b>2 Requirements Analysis and Specification</b>      | <b>11</b> |
| 2.1 Introduction . . . . .                            | 12        |
| 2.2 Requirements Specification . . . . .              | 12        |
| 2.2.1 Stakeholders Identification . . . . .           | 12        |
| 2.2.2 Functional Requirements Specification . . . . . | 12        |
| 2.2.3 Non-Functional Requirements . . . . .           | 13        |
| 2.3 Global Use Case Diagram . . . . .                 | 14        |
| 2.4 Class Diagram . . . . .                           | 16        |
| 2.5 Project Structure and Breakdown . . . . .         | 16        |
| 2.5.1 Scrum Team Identification . . . . .             | 16        |
| 2.5.2 Product Backlog . . . . .                       | 17        |

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| 2.5.3    | Sprint Planning . . . . .                                  | 20        |
| 2.6      | Application Architecture . . . . .                         | 23        |
| 2.6.1    | Physical Architecture . . . . .                            | 23        |
| 2.6.2    | Logical Architecture . . . . .                             | 24        |
| 2.7      | Technical Choices and Development Environment . . . . .    | 25        |
| 2.7.1    | Programming Languages . . . . .                            | 26        |
| 2.7.2    | API Layer . . . . .                                        | 28        |
| 2.7.3    | Frameworks . . . . .                                       | 28        |
| 2.7.4    | Database . . . . .                                         | 30        |
| 2.7.5    | Observability . . . . .                                    | 31        |
| 2.7.6    | Development Tools . . . . .                                | 33        |
| 2.8      | Conclusion . . . . .                                       | 36        |
| <b>3</b> | <b>Release 1 : MVP</b>                                     | <b>37</b> |
| 3.1      | Introduction . . . . .                                     | 38        |
| 3.2      | Release 1 Backlog . . . . .                                | 38        |
| 3.3      | Refined Use Case Diagram for Release 1 . . . . .           | 40        |
| 3.4      | Class Diagram of Release 1 . . . . .                       | 40        |
| 3.5      | Sprint 1 : Authentication and User Foundations . . . . .   | 41        |
| 3.5.1    | Sprint 1 Objectives . . . . .                              | 41        |
| 3.5.2    | Refined Use Case Diagram for Sprint 1 . . . . .            | 41        |
| 3.5.3    | Analysis and Design for Sprint 1 . . . . .                 | 42        |
| 3.5.4    | Realization of Sprint 1 . . . . .                          | 46        |
| 3.6      | Sprint 2 : Mission Management . . . . .                    | 48        |
| 3.6.1    | Sprint 2 Objectives . . . . .                              | 48        |
| 3.6.2    | Refined Use Case Diagram for Sprint 2 . . . . .            | 48        |
| 3.6.3    | Analysis and Design for Sprint 2 . . . . .                 | 48        |
| 3.6.4    | Realization of Sprint 2 . . . . .                          | 52        |
| 3.7      | Sprint 3 : Station Monitoring and Data Ingestion . . . . . | 54        |
| 3.7.1    | Sprint 3 Objectives . . . . .                              | 54        |
| 3.7.2    | Refined Use Case Diagram for Sprint 3 . . . . .            | 54        |
| 3.7.3    | Analysis and Design for Sprint 3 . . . . .                 | 54        |
| 3.7.4    | Realization of Sprint 3 . . . . .                          | 58        |
| 3.8      | Conclusion . . . . .                                       | 60        |

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>4 Release 2 : Improvements</b>                          | <b>61</b> |
| 4.1 Introduction . . . . .                                 | 62        |
| 4.2 Release 2 Backlog . . . . .                            | 62        |
| 4.3 Refined Use Case Diagram for Release 2 . . . . .       | 63        |
| 4.4 Class Diagram of Release 2 . . . . .                   | 64        |
| 4.5 Sprint 4 : Supervision Dashboard . . . . .             | 65        |
| 4.5.1 Sprint 4 Objectives . . . . .                        | 65        |
| 4.5.2 Refined Use Case Diagram for Sprint 4 . . . . .      | 65        |
| 4.5.3 Analysis and Design for Sprint 4 . . . . .           | 66        |
| 4.5.4 Realization of Sprint 4 . . . . .                    | 70        |
| 4.6 Sprint 5 : Alerts & Notifications . . . . .            | 72        |
| 4.6.1 Sprint 5 Objectives . . . . .                        | 72        |
| 4.6.2 Refined Use Case Diagram for Sprint 5 . . . . .      | 72        |
| 4.6.3 Analysis and Design for Sprint 5 . . . . .           | 72        |
| 4.6.4 Realization of Sprint 5 . . . . .                    | 76        |
| 4.7 Sprint 6 : Advanced Features and Scalability . . . . . | 78        |
| 4.7.1 Sprint 6 Objectives . . . . .                        | 78        |
| 4.7.2 Refined Use Case Diagram for Sprint 6 . . . . .      | 78        |
| 4.7.3 Analysis and Design for Sprint 6 . . . . .           | 78        |
| 4.7.4 Realization of Sprint 6 . . . . .                    | 82        |
| 4.8 Conclusion . . . . .                                   | 84        |
| <b>General Conclusion</b>                                  | <b>85</b> |
| <b>Webography</b>                                          | <b>87</b> |

---

# List of Figures

|      |                                              |    |
|------|----------------------------------------------|----|
| 1.1  | Logo ENtityservice . . . . .                 | 4  |
| 1.2  | EntityService Organizational Chart . . . . . | 5  |
| 1.3  | SCRUM Process . . . . .                      | 9  |
| 1.4  | Logo UML <b>uml_logo</b> . . . . .           | 10 |
| 2.1  | Global Use Case Diagram . . . . .            | 15 |
| 2.2  | Global Class Diagram . . . . .               | 16 |
| 2.3  | Physical Architecture (N-Tier) . . . . .     | 24 |
| 2.4  | Logical Architecture (MVC) . . . . .         | 25 |
| 2.5  | Go Programming Language Logo . . . . .       | 26 |
| 2.6  | HTML5 Logo . . . . .                         | 26 |
| 2.7  | CSS3 Logo . . . . .                          | 27 |
| 2.8  | JavaScript Logo . . . . .                    | 27 |
| 2.9  | TypeScript Logo . . . . .                    | 27 |
| 2.10 | GraphQL Logo . . . . .                       | 28 |
| 2.11 | Gin Framework Logo . . . . .                 | 28 |
| 2.12 | gqlgen Logo . . . . .                        | 29 |
| 2.13 | React Logo . . . . .                         | 29 |
| 2.14 | React Native Logo . . . . .                  | 29 |
| 2.15 | Apollo Client Logo . . . . .                 | 30 |
| 2.16 | PostgreSQL Logo . . . . .                    | 30 |
| 2.17 | TimescaleDB Logo . . . . .                   | 31 |
| 2.18 | Redis Logo . . . . .                         | 31 |
| 2.19 | Elasticsearch Logo . . . . .                 | 31 |
| 2.20 | Logstash Logo . . . . .                      | 32 |
| 2.21 | Kibana Logo . . . . .                        | 32 |
| 2.22 | Prometheus Logo . . . . .                    | 33 |
| 2.23 | Grafana Logo . . . . .                       | 33 |
| 2.24 | GoLand IDE Logo . . . . .                    | 33 |
| 2.25 | Visual Studio Code Logo . . . . .            | 34 |
| 2.26 | Postman Logo . . . . .                       | 34 |
| 2.27 | Docker Logo . . . . .                        | 35 |

|      |                                                          |    |
|------|----------------------------------------------------------|----|
| 2.28 | Git Logo . . . . .                                       | 35 |
| 2.29 | GitHub Logo . . . . .                                    | 35 |
| 3.1  | Refined Use Case Diagram – Release 1 . . . . .           | 40 |
| 3.2  | Class Diagram – Release 1 . . . . .                      | 41 |
| 3.3  | Refined Use Case Diagram – Sprint 1 . . . . .            | 42 |
| 3.4  | Sequence Diagram – Login . . . . .                       | 44 |
| 3.5  | Sequence Diagram – Add User . . . . .                    | 46 |
| 3.6  | Sprint 1 – Login Interface . . . . .                     | 47 |
| 3.7  | Sprint 1 – User Management Interface . . . . .           | 47 |
| 3.8  | Refined Use Case Diagram – Sprint 2 . . . . .            | 48 |
| 3.9  | Sequence Diagram – Create and Assign Mission . . . . .   | 50 |
| 3.10 | Sequence Diagram – Update Mission Status . . . . .       | 52 |
| 3.11 | Sprint 2 – Mission Management Interface (Web) . . . . .  | 53 |
| 3.12 | Sprint 2 – Technician Mobile Application . . . . .       | 53 |
| 3.13 | Refined Use Case Diagram – Sprint 3 . . . . .            | 54 |
| 3.14 | Sequence Diagram – Automated Data Collection . . . . .   | 56 |
| 3.15 | Sequence Diagram – Monitor Collector Health . . . . .    | 58 |
| 3.16 | Sprint 3 – Data Collector Monitoring Interface . . . . . | 59 |
| 3.17 | Sprint 3 – Ingestion Logs Interface . . . . .            | 59 |
| 4.1  | Refined Use Case Diagram – Release 2 . . . . .           | 64 |
| 4.2  | Class Diagram – Release 2 . . . . .                      | 65 |
| 4.3  | Refined Use Case Diagram – Sprint 4 . . . . .            | 66 |
| 4.4  | Sequence Diagram – Monitor Station Fleet . . . . .       | 68 |
| 4.5  | Sequence Diagram – Consult Historical Data . . . . .     | 70 |
| 4.6  | Sprint 4 – Web Supervision Dashboard . . . . .           | 71 |
| 4.7  | Sprint 4 – Historical Data Visualization . . . . .       | 71 |
| 4.8  | Refined Use Case Diagram – Sprint 5 . . . . .            | 72 |
| 4.9  | Sequence Diagram – Receive Real-Time Alert . . . . .     | 74 |
| 4.10 | Sequence Diagram – Push Notification . . . . .           | 76 |
| 4.11 | Sprint 5 – Alert Management Interface . . . . .          | 77 |
| 4.12 | Sprint 5 – Push Notification on Mobile . . . . .         | 77 |
| 4.13 | Refined Use Case Diagram – Sprint 6 . . . . .            | 78 |
| 4.14 | Sequence Diagram – AI-Based Anomaly Detection . . . . .  | 80 |

|                                                          |    |
|----------------------------------------------------------|----|
| 4.15 Sequence Diagram – Provision New Tenant . . . . .   | 82 |
| 4.16 Sprint 6 – AI Anomaly Detection Dashboard . . . . . | 83 |
| 4.17 Sprint 6 – Multi-Tenant Management Portal . . . . . | 83 |

# List of Tables

|     |                                                                       |    |
|-----|-----------------------------------------------------------------------|----|
| 2.1 | Product Backlog . . . . .                                             | 20 |
| 2.2 | Sprint Planning . . . . .                                             | 23 |
| 3.1 | Release 1 Backlog (MVP) . . . . .                                     | 40 |
| 3.2 | Textual Description – Use Case «Authenticate» . . . . .               | 43 |
| 3.3 | Textual Description – Use Case «User Management» . . . . .            | 45 |
| 3.4 | Textual Description – Use Case «Create and Assign Mission» . . . . .  | 49 |
| 3.5 | Textual Description – Use Case «Update Mission Status» . . . . .      | 51 |
| 3.6 | Textual Description – Use Case «Automated Data Collection» . . . . .  | 55 |
| 3.7 | Textual Description – Use Case «Monitor Collector Health» . . . . .   | 57 |
| 4.1 | Release 2 Backlog (Improvements) . . . . .                            | 63 |
| 4.2 | Textual Description – Use Case «Monitor Station Fleet» . . . . .      | 67 |
| 4.3 | Textual Description – Use Case «Consult Historical Data» . . . . .    | 69 |
| 4.4 | Textual Description – Use Case «Receive Real-Time Alert» . . . . .    | 73 |
| 4.5 | Textual Description – Use Case «Push Notification» . . . . .          | 75 |
| 4.6 | Textual Description – Use Case «AI-Based Anomaly Detection» . . . . . | 79 |
| 4.7 | Textual Description – Use Case «Provision New Tenant» . . . . .       | 81 |

---

# List of Abbreviations

| Abbreviation | Description                                    |
|--------------|------------------------------------------------|
| API          | Application Programming Interface              |
| ACID         | Atomicity, Consistency, Isolation, Durability  |
| AI           | Artificial Intelligence                        |
| CSS          | Cascading Style Sheets                         |
| CRUD         | Create, Read, Update, Delete                   |
| CI/CD        | Continuous Integration / Continuous Deployment |
| DB           | Database                                       |
| DOM          | Document Object Model                          |
| ELK          | Elasticsearch, Logstash, Kibana                |
| EV           | Electric Vehicle                               |
| FCM          | Firebase Cloud Messaging                       |
| FK           | Foreign Key                                    |
| GQL          | GraphQL                                        |
| GORM         | Go Object Relational Mapper                    |
| HTML         | HyperText Markup Language                      |
| HTTP         | HyperText Transfer Protocol                    |
| HTTPS        | HyperText Transfer Protocol Secure             |
| IDE          | Integrated Development Environment             |
| IT           | Information Technology                         |
| JS           | JavaScript                                     |
| JSON         | JavaScript Object Notation                     |
| JWT          | JSON Web Token                                 |
| MoSCoW       | Must Have, Should Have, Could Have, Won't Have |
| MVC          | Model View Controller                          |
| MVP          | Minimum Viable Product                         |
| OCPP         | Open Charge Point Protocol                     |
| OOP          | Object Oriented Programming                    |
| ORM          | Object Relational Mapper                       |
| OS           | Operating System                               |

| Abbreviation | Description                     |
|--------------|---------------------------------|
| PK           | Primary Key                     |
| REST         | Representational State Transfer |
| SaaS         | Software as a Service           |
| SDL          | Schema Definition Language      |
| SQL          | Structured Query Language       |
| SSL          | Secure Sockets Layer            |
| TLS          | Transport Layer Security        |
| TS           | TypeScript                      |
| UI           | User Interface                  |
| UML          | Unified Modeling Language       |
| URL          | Uniform Resource Locator        |
| UUID         | Universally Unique Identifier   |
| UX           | User Experience                 |
| W3C          | World Wide Web Consortium       |
| WS           | WebSocket                       |

---

# General Introduction

The global transition toward sustainable mobility has accelerated the deployment of electric vehicles at an unprecedented pace. As the number of electric vehicles on the road continues to grow, the infrastructure supporting them – charging stations – must scale accordingly in terms of quantity, reliability, and operational efficiency. Managing a growing fleet of charging stations, however, presents significant operational challenges that traditional tools such as spreadsheets are no longer equipped to handle.

In this context, EntityService, a company specializing in the design and development of innovative digital solutions, identified the need for a modern, integrated platform capable of replacing its client’s manual and fragmented workflow. The existing approach, based on Google Sheets, suffered from a lack of real-time automation, limited scalability, poor traceability, and an absence of structured tools for field technicians. These limitations directly impacted the quality of service delivered to the client and the efficiency of the teams responsible for station maintenance and supervision.

The objective of this project is to design and develop an integrated system composed of three core components: an automatic data collector that continuously ingests real-time data from EV charging stations, a mobile application dedicated to field technicians for managing their intervention missions, and a web supervision dashboard that provides back-office teams with a comprehensive, real-time view of the entire station fleet.

The system is built on a modern and robust technical stack, combining the performance of the **Go** programming language for the backend and data collection layer, **GraphQL** as the unified API layer, **React** for the web supervision dashboard, and **React Native** for the cross-platform mobile application. Data persistence relies on **PostgreSQL** and **TimescaleDB**, while observability is ensured through the **ELK** stack combined with **Prometheus** and **Grafana**.

The development process follows the **Scrum** agile methodology, organized into two releases and five sprints, ensuring iterative delivery of value and continuous alignment with the client’s evolving requirements.

This report is structured as follows:

- **Chapter 1** presents the project context, the host company EntityService, the limitations of the existing system, and the proposed solution. It also introduces the Scrum methodology and the UML modeling language adopted throughout the project.
- **Chapter 2** covers the analysis and specification of requirements. It identifies the system’s actors, defines the functional and non-functional requirements, establishes the product

- 
- backlog, details the sprint planning across two releases, presents the system architecture at both physical and logical levels, and justifies the technical choices made for the project.
- **Chapter 3** will detail the design and implementation of Release 1, which will contain 3 sprints.
  - **Chapter 4** focus on Release 2, where we will present the different functionalities of sprints 4 and 5, and the analysis and implementation of each sprint.

This work represents a concrete step toward the digitalization and professionalization of EV charging station management, laying the foundation for a scalable, reliable, and future-ready platform.

---

# PROJECT CONTEXT AND OBJECTIVES

---

## Plan

|   |                                            |    |
|---|--------------------------------------------|----|
| 1 | Introduction . . . . .                     | 4  |
| 2 | Presentation of the Host Company . . . . . | 4  |
| 3 | Project Presentation . . . . .             | 5  |
| 4 | Project Management Methodology . . . . .   | 7  |
| 5 | Modeling Language . . . . .                | 9  |
| 6 | Conclusion . . . . .                       | 10 |

## 1.1 Introduction

This chapter presents the context of the project, the host company, the problem statement, and the methodology adopted for the development of the mobile application for managing electric vehicle charging stations.

## 1.2 Presentation of the Host Company

### 1.2.1 EntityService Presentation

EntityService is a consulting firm specializing in digital strategy and innovative IT solutions. Recognized for its expertise in digital transformation, the company supports businesses in defining and implementing their digital strategies by delivering tailored solutions that address their specific needs and help them stay competitive in an ever-evolving technological landscape.



**Figure 1.1:** Logo ENTityservice

### 1.2.2 Mission and Vision of the Company

**Mission:** EntityService’s mission is to serve as a trusted partner for companies seeking to optimize their operations through technology. As a consulting firm in digital strategy and innovative solutions development, the company is committed to designing and delivering customized IT solutions that align with each client’s unique objectives, while upholding the values of excellence, innovation, quality, and responsibility.

**Vision:** EntityService aspires to become the partner of choice for organizations across diverse industries, empowering them to reach their full potential through responsible and forward-thinking technological solutions. By positioning itself at the forefront of digital transformation, the company aims to act as a true catalyst for innovation and progress — creating lasting value for its clients, collaborators, and society as a whole.

### 1.2.3 Organizational Chart

Figure 1.2 illustrates the organizational chart of Entity Service

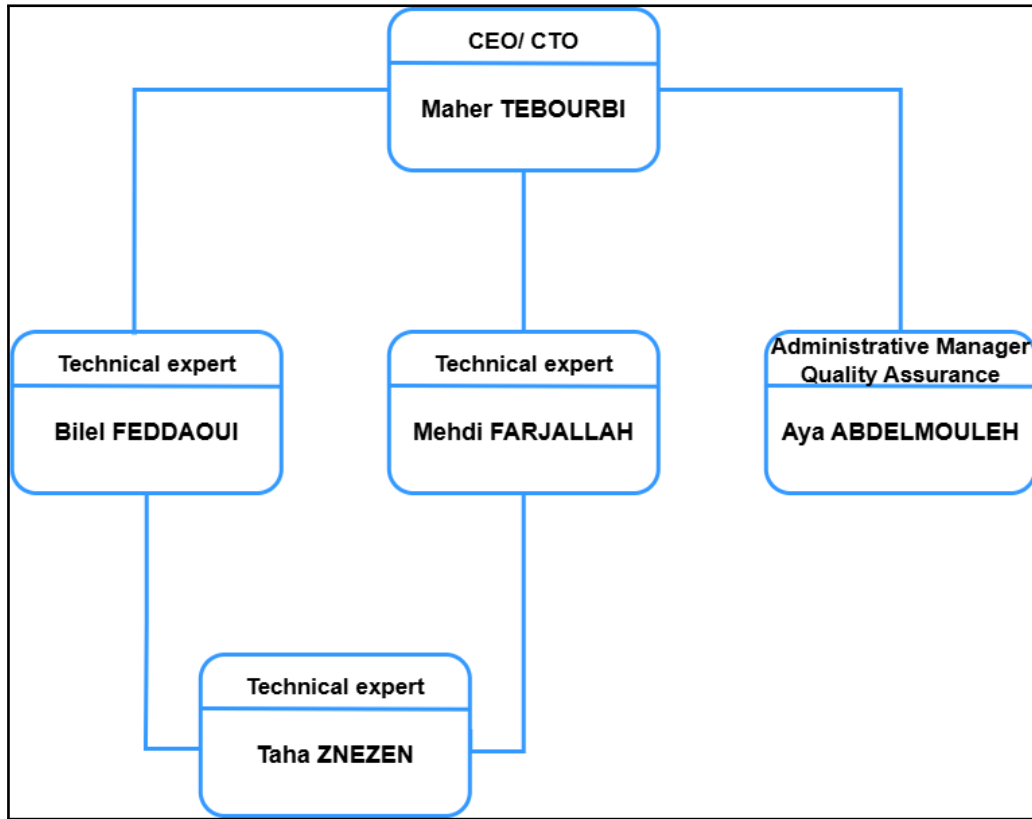


Figure 1.2: EntityService Organizational Chart

## 1.3 Project Presentation

To successfully carry out this project, a preliminary study was conducted to describe the existing system and identify its limitations, in order to develop a solution that best meets our client’s needs.

### 1.3.1 Existing Systems Study

EntityService client currently relies on Google Sheets to manage technician missions and collect data from EV charging stations. While this approach proved functional at a small scale, several integrated digital platforms exist on the market that attempt to address similar needs, including:

- **ChargePoint** – a cloud-based EV station management platform offering real-time monitoring and fleet management dashboards;
- **OCPP-based CPMS solutions** (e.g., EV-Box, Siemens eMobility) – platforms using the Open Charge Point Protocol for centralized station supervision;
- **Generic field-service management tools** (e.g., ServiceMax, Praxedo) – mobile-first applications designed for technician mission tracking and scheduling.

However, none of these solutions fully address the specific operational needs of EntityService client, particularly the combined requirement of real-time EV data collection and integrated technician mission management.

### 1.3.2 Critique of Existing Systems

The analysis of existing solutions reveals several recurring limitations in the current context:

- **Lack of integration:** Generic mission management tools do not natively interface with EV charging station protocols or APIs, requiring costly custom development or manual data entry;
- **Absence of real-time automation:** The current Google Sheets workflow lacks automatic, reliable real-time updates, leading to data inconsistencies and significant error risks during manual entry or copy operations;
- **Limited scalability:** As the fleet of charging stations grows, spreadsheet-based supervision becomes increasingly difficult to maintain, with no support for structured alerts, role-based access, or audit trails;
- **No traceability or validation:** Existing solutions do not offer automatic validation rules, anomaly detection, or centralized logging of critical modifications, which undermines data reliability and accountability;
- **Poor offline and mobility support:** Technicians operating in the field lack a dedicated mobile interface, forcing reliance on generic communication tools with no synchronization or geolocation capabilities.

### 1.3.3 Proposed Solution

To overcome the limitations identified above, EntityService client proposes the development of a fully integrated and specialized system composed of three main components:

- **An Automatic Data Collector** – a dedicated service that connects securely and continuously to EV charging stations to retrieve their status, measurements, signals, and alerts in real time. Data is automatically validated, normalized, and centralized in a dedicated server, eliminating all manual entry;
- **A Mobile Application for Technicians** – an intuitive and responsive mobile interface allowing field technicians to authenticate securely, view and manage their assigned missions, update statuses in real time (in progress, completed, incident), and operate in offline mode with automatic synchronization upon reconnection;
- **A Web Supervision Application** – a powerful, modern web dashboard enabling back-office teams to monitor the entire station fleet and ongoing missions in real time, apply filters and search criteria, consult historical data, and configure a flexible alert system.

This architecture replaces the fragmented Google Sheets-based workflow with a reliable, secure,

and scalable infrastructure. It ensures end-to-end traceability, reduces human error significantly, and positions EntityService client for future growth through planned evolutions such as predictive analytics, AI-based anomaly detection, and a multi-fleet SaaS model.

## 1.4 Project Management Methodology

Agility represents a dynamic and adaptive approach to project management and software development. It emphasizes flexibility, collaboration, and the ability to adjust to changing requirements rather than strictly following a predefined plan. This methodology is based on short cycles, called iterations, which promote continuous improvement and ensure rapid delivery of functional and high-quality products.

Among Agile methods, **Scrum** is one of the most widely used. It is a simple yet powerful framework designed to enable effective team collaboration while responding to user needs. Through its well-defined roles, events, and artifacts, Scrum supports the creation of high-quality products and encourages team engagement and shared responsibility.

### 1.4.1 Scrum Roles

Scrum defines three main roles:

**Product Owner (PO)** The Product Owner is the primary representative of stakeholders and customers. They are responsible for prioritizing the Product Backlog items to ensure that the development team focuses on the most valuable features. By collecting feedback from users, the PO ensures that the product meets real market needs.

**Scrum Master** The Scrum Master acts as the facilitator of the Scrum process. They help the team follow Agile principles, remove obstacles, and improve continuously. The Scrum Master fosters a collaborative and productive work environment.

**Development Team** The development team is composed of multidisciplinary professionals who design, develop, and deliver product functionality at the end of each Sprint. The team is autonomous and collaborates closely to transform backlog items into tangible results while maintaining quality standards.

### 1.4.2 Scrum Events

Scrum includes several key events that structure the development process.

**Sprint** A Sprint is an iteration lasting between one and four weeks during which the team commits to delivering a functional increment of the product. Each Sprint begins with planning and ends with an evaluation to ensure continuous progress.

**Daily Scrum** The Daily Scrum is a 15-minute daily meeting that allows the team to synchronize, discuss progress, identify obstacles, and plan upcoming tasks. This event enhances transparency and communication.

**Sprint Planning** During Sprint Planning, the team selects the Product Backlog items to be completed during the Sprint. This event clarifies the Sprint Goal and ensures that all members understand the required work.

**Sprint Review** At the end of each Sprint, the completed work is presented to stakeholders. The Sprint Review provides valuable feedback and helps refine the priorities of the Product Backlog.

**Sprint Retrospective** The Sprint Retrospective focuses on continuous improvement. The team reflects on what went well, what could be improved, and defines actions to implement in the next Sprint.

### 1.4.3 Scrum Artifacts

**Product Backlog** The Product Backlog is a prioritized list of features, enhancements, and requirements. The most important items appear first. The Product Owner manages the backlog, while the development team selects the work based on its capacity.

**Sprint Backlog** The Sprint Backlog is a subset of the Product Backlog selected for a specific Sprint. It contains the chosen items and a detailed plan for completing them. The Sprint Backlog may evolve during the Sprint as the team works toward the Sprint Goal.

**Increment** The Increment is composed of all completed Product Backlog items from the Sprint, combined with the value of previous increments. It must meet the Definition of Done and be in a potentially releasable state.

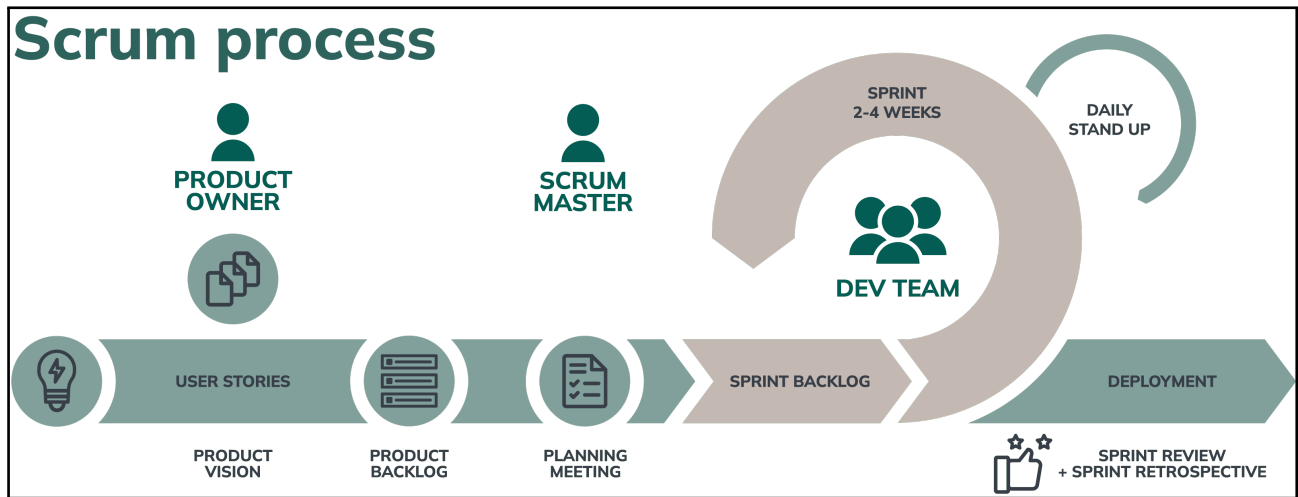


Figure 1.3: SCRUM Process

## 1.5 Modeling Language

To ensure clarity, consistency, and effective communication between all stakeholders throughout the development process, the project adopts **UML (Unified Modeling Language)** as it is widely recognized for its ability to visually and semantically represent the structure and behavior of complex software systems.

UML is a modeling language, not a method – meaning the use of its diagrams is left to the discretion of each team. The current version, **UML 2.5**, offers **14 types of diagrams**, divided into seven structural and seven behavioral categories [1]. Methods such as the Unified Process, proposed by the original creators of UML, make more systematic use of the full diagram set and center analysis on use cases to iteratively develop analysis models, design models, and others [2]. Other approaches limit themselves to modeling only certain critical parts of a system that are difficult to infer directly from code [3].



**Figure 1.4:** Logo UML `uml_logo`

## 1.6 Conclusion

In this first chapter, we presented the context of the project, identified the limitations of the client's current workflow, and introduced the proposed solution. We also justified the choice of the Scrum methodology and the UML modeling language adopted throughout this project. The following chapter will proceed with the analysis and specification of the project requirements.

# REQUIREMENTS ANALYSIS AND SPECIFICATION

---

## Plan

|   |                                                         |    |
|---|---------------------------------------------------------|----|
| 1 | Introduction . . . . .                                  | 12 |
| 2 | Requirements Specification . . . . .                    | 12 |
| 3 | Global Use Case Diagram . . . . .                       | 14 |
| 4 | Class Diagram . . . . .                                 | 16 |
| 5 | Project Structure and Breakdown . . . . .               | 16 |
| 6 | Application Architecture . . . . .                      | 23 |
| 7 | Technical Choices and Development Environment . . . . . | 25 |
| 8 | Conclusion . . . . .                                    | 36 |

## 2.1 Introduction

This chapter presents the analysis and specification of requirements for the integrated EV charging station management system commissioned by EntityService’s client. It covers the identification of stakeholders, functional and non-functional requirements, and serves as the foundation for the UML modeling and architectural choices presented in the following sections.

## 2.2 Requirements Specification

In this section, we will identify the application’s stakeholders, as well as its functional and non-functional requirements.

### 2.2.1 Stakeholders Identification

An actor represents any person, team, or entity that has an interest, influence, or impact on the developed project or product. The following actors have been identified for this system:

- **Supervisor:** Responsible for monitoring the entire fleet of EV charging stations and overseeing technician missions in real time through the web supervision dashboard;
- **Technician:** A field operator who performs on-site maintenance and intervention missions. He interacts with the system exclusively through the mobile application to receive, update, and close his assigned missions;
- **Administrator:** The platform moderator, responsible for managing, supervising, and maintaining the application and its database, ensuring its correct operation and security;

### 2.2.2 Functional Requirements Specification

We specify the functional requirements per actor.

#### 2.2.2.1 Technician

- Authenticate securely to the mobile application;
- Access the list of assigned missions and consult their details (location, description, priority, deadline);
- Update mission status in real time (pending, in progress, completed, incident);
- Consult personal intervention history;
- Operate in offline mode with automatic data synchronization upon reconnection;
- Optionally capture geolocation data at the time of intervention.

### 2.2.2.2 Supervisor

- Access the real-time supervision dashboard;
- Consult the global status of the EV station fleet and all active missions;
- View detailed information per station (operational, faulty, offline, under maintenance);
- Search, filter, sort, and export station and mission data;
- Consult historical data and intervention records;
- Receive and configure alerts for critical events (station failure, missed mission, data anomaly).

### 2.2.2.3 Administrator

- Access the administration dashboard;
- Manage user accounts: create, modify, and deactivate technician and supervisor accounts;
- Manage charging station records within the platform;
- Consult system usage statistics and audit logs;
- Ensure platform security, role assignments, and access control.

## 2.2.3 Non-Functional Requirements

Non-functional requirements refer to the criteria that define the quality and constraints of a system. They act indirectly on the system's output and on user productivity.

- **Ergonomics:** The mobile application and web dashboard must provide simple, intuitive interfaces that require minimal training for both technicians and supervisors;
- **Responsiveness:** The web application must deliver an optimal experience across a wide range of devices and screen resolutions, adapting its layout accordingly;
- **Performance:** The system must reflect station data updates within 3 seconds of ingestion; the mobile application must remain fluid and reactive even under poor network conditions;
- **Availability:** The data collector and backend server must maintain a 99.9% uptime to ensure continuous and uninterrupted station monitoring;
- **Scalability:** The architecture must support a growing fleet of stations and an increasing number of concurrent users without performance degradation;
- **Security and Confidentiality:** The system must guarantee full protection of personal and operational data. All data flows must be encrypted via TLS, authentication enforced through JWT or OAuth2, and access strictly limited to authorized users. Passwords must never be stored or transmitted in plain text;
- **Traceability:** All mission updates, data modifications, and critical access events must be

logged and auditable at all times;

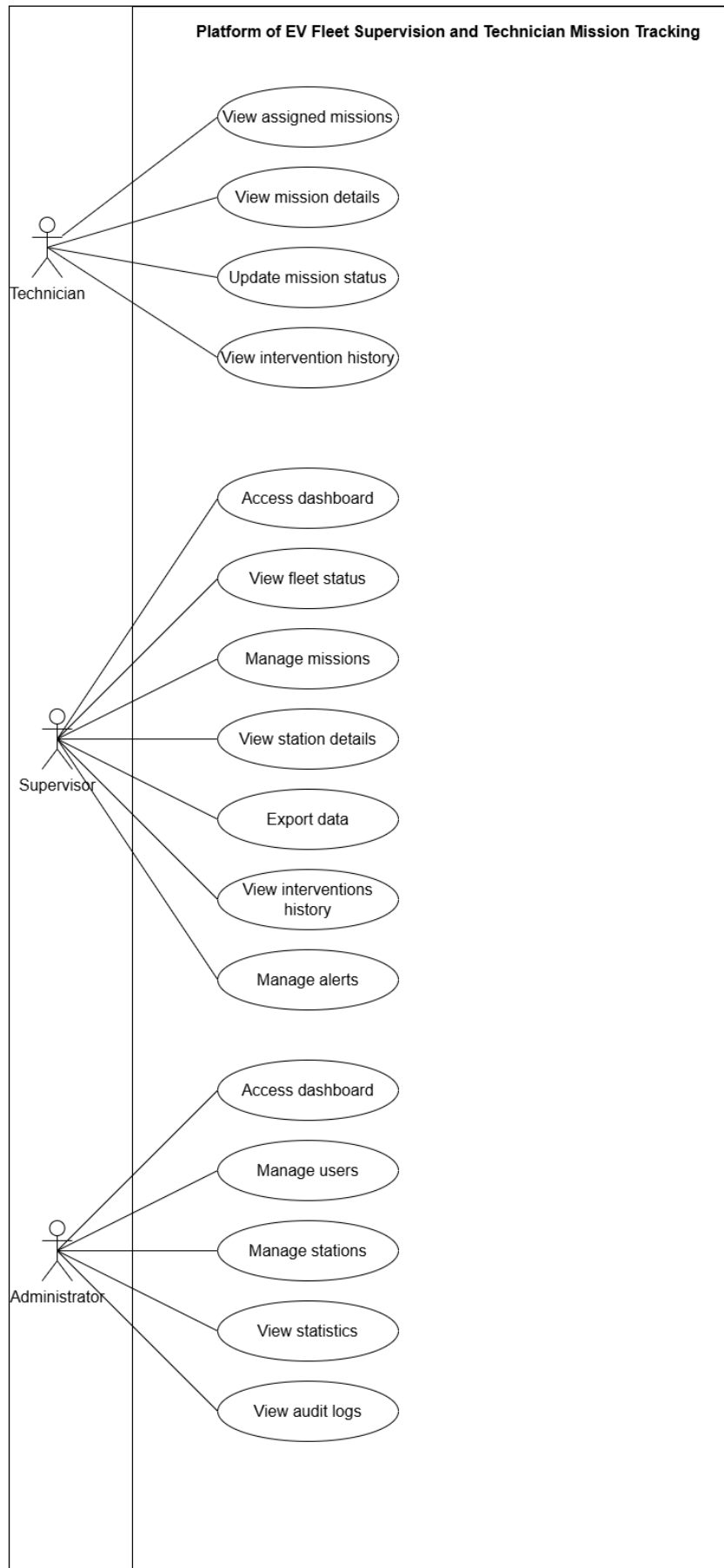
- **Maintainability and Evolvability:** The system must be designed with a flexible and extensible architecture, capable of accommodating future evolutions such as AI-based anomaly detection, predictive maintenance, and multi-fleet SaaS support.

## 2.3 Global Use Case Diagram

Use case diagrams identify the functionalities provided by the system (use cases), the users who interact with the system (actors), and the interaction between these actors. The main objectives of use case diagrams are: Figure 2.1 shows the overall use case diagram:

Note: All functionalities, require authentication.

- **Use cases:** provide a high-level view of what the system does.
- **actors:** identify the users of the system.

**Figure 2.1:** Global Use Case Diagram

## 2.4 Class Diagram

Class diagram is a static Unified Modeling Language (UML) diagram that models the structure of a system by mapping out its classes, attributes, operations (methods), and relationships.

Figure 2.2 shows our class diagram.

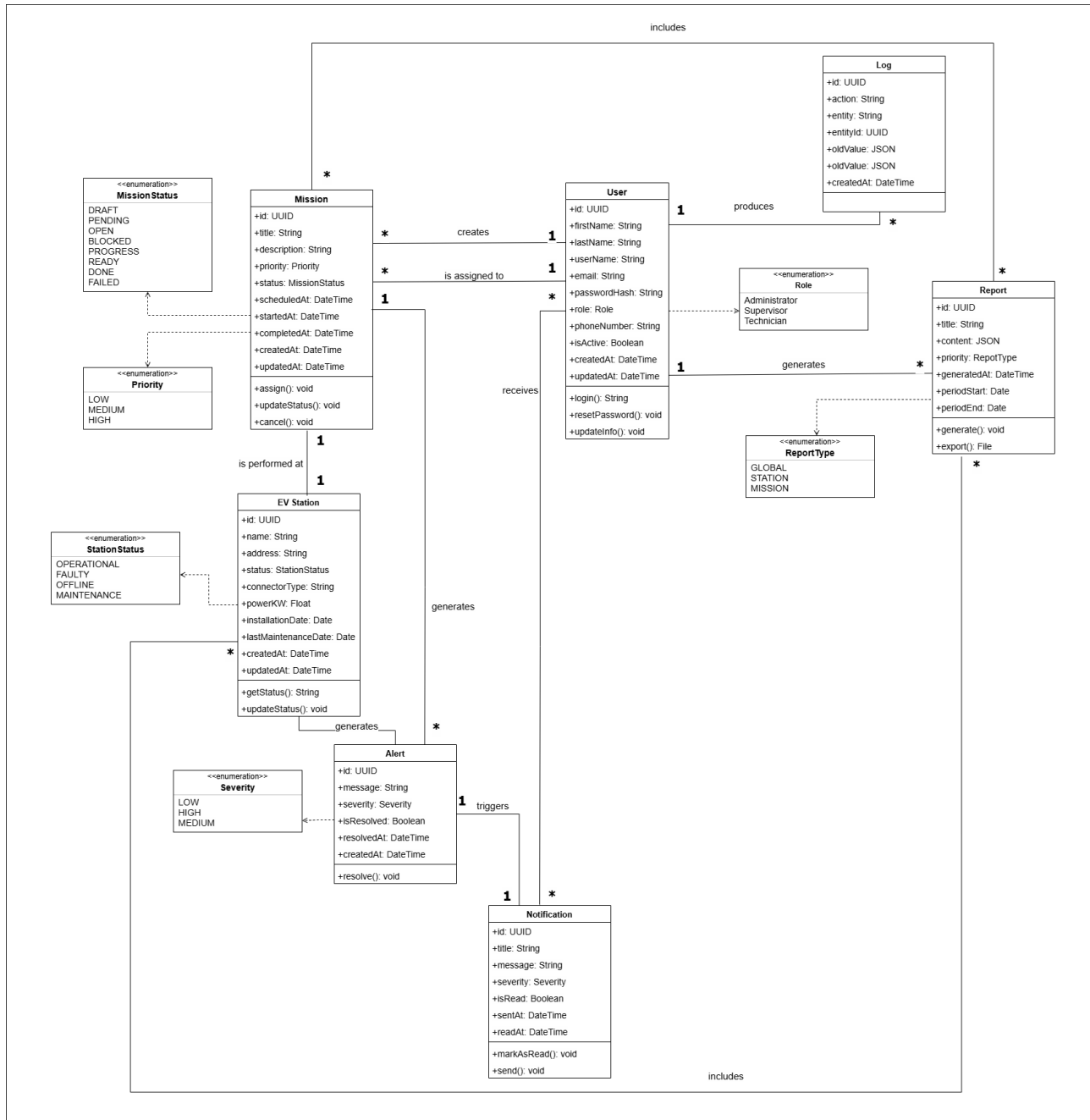


Figure 2.2: Global Class Diagram

## 2.5 Project Structure and Breakdown

### 2.5.1 Scrum Team Identification

The Scrum team for this project consists of:

- **Product Owner:** Maher Tebourbi
- **Scrum Master:** Raouf Abrougui
- **Developer:** Rayen Aouechria

### 2.5.2 Product Backlog

Table 2.1 represents the Product Backlog of our application. **Legend**

#### MoSCoW Priority:

|                             |                                         |
|-----------------------------|-----------------------------------------|
| <b>Must - Must have</b>     | Essential feature, required for the MVP |
| <b>Should - Should have</b> | Important feature but not critical      |
| <b>Could - Could have</b>   | Desirable feature but not necessary     |
| <b>Won't - Won't have</b>   | Feature not included in this version    |

#### Complexity:

|   |                                                         |
|---|---------------------------------------------------------|
| 1 | Simple - Minimal effort, straightforward implementation |
| 2 | Moderate - Moderate effort, average complexity          |
| 3 | Complex - Significant effort, complex implementation    |
| 5 | Very complex - High effort, high complexity             |

| ID | Feature                                 | US ID | User Story                                                                            | Priority | Complexi |
|----|-----------------------------------------|-------|---------------------------------------------------------------------------------------|----------|----------|
| 1  | Authentication<br>& User<br>Foundations | 1.1   | As a user, I want to log in securely to the system.                                   | Must     | 1        |
|    |                                         | 1.2   | As a user, I want to reset my password via a secure process.                          | Should   | 2        |
|    |                                         | 1.3   | As an administrator, I want role-based access control enforced across all components. | Must     | 2        |
|    |                                         | 1.4   | As an administrator, I want to create technician and supervisor accounts.             | Must     | 2        |

*Continued on next page...*

| ID | Feature                           | US ID | User Story                                                                                                                          | Priority | Complexi |
|----|-----------------------------------|-------|-------------------------------------------------------------------------------------------------------------------------------------|----------|----------|
|    |                                   | 1.5   | As an administrator, I want to modify and deactivate user accounts.                                                                 | Must     | 2        |
|    |                                   | 1.6   | As an administrator, I want to manage role assignments and access permissions.                                                      | Must     | 2        |
|    |                                   | 1.7   | As an administrator, I want to consult system usage statistics and audit logs.                                                      | Should   | 3        |
| 2  | Mission Management (Web & Mobile) | 2.1   | As a supervisor, I want to create and assign intervention missions to technicians.                                                  | Must     | 3        |
|    |                                   | 2.2   | As a supervisor, I want to track the lifecycle of each mission in real time (pending, in progress, completed, incident).            | Must     | 3        |
|    |                                   | 2.3   | As a supervisor, I want to consult the full intervention history per station and per technician.                                    | Should   | 2        |
|    |                                   | 2.4   | As an administrator, I want to manage mission types and assignment rules.                                                           | Could    | 2        |
|    |                                   | 2.5   | As a technician, I want to view the list of missions assigned to me with their details (location, description, priority, deadline). | Must     | 2        |
|    |                                   | 2.6   | As a technician, I want to update my mission status in real time with minimal interaction.                                          | Must     | 2        |
|    |                                   | 2.7   | As a technician, I want to use the application in offline mode with automatic synchronization upon reconnection.                    | Must     | 3        |
|    |                                   | 2.8   | As a technician, I want to consult my personal intervention history.                                                                | Should   | 1        |

Continued on next page...

| ID | Feature                             | US ID | User Story                                                                                                          | Priority | Complexi |
|----|-------------------------------------|-------|---------------------------------------------------------------------------------------------------------------------|----------|----------|
|    |                                     | 2.9   | As a technician, I want my geolocation to be optionally captured at the time of intervention.                       | Could    | 2        |
| 3  | Station Monitoring & Data Ingestion | 3.1   | As a supervisor, I want the system to automatically connect to EV charging stations and retrieve their data.        | Must     | 3        |
|    |                                     | 3.2   | As a supervisor, I want station data (status, measurements, alerts) to be ingested continuously in real time.       | Must     | 3        |
|    |                                     | 3.3   | As a supervisor, I want incoming data to be validated and anomalies handled automatically.                          | Must     | 2        |
|    |                                     | 3.4   | As a supervisor, I want collected data to be normalized and stored in a structured, unified format.                 | Must     | 2        |
|    |                                     | 3.5   | As an administrator, I want to consult collection logs and monitor the collector's health.                          | Should   | 2        |
| 4  | Supervision Dashboard               | 4.1   | As a supervisor, I want to access a real-time dashboard showing the status of all EV charging stations.             | Must     | 3        |
|    |                                     | 4.2   | As a supervisor, I want to view detailed information per station (operational, faulty, offline, under maintenance). | Must     | 2        |
|    |                                     | 4.3   | As a supervisor, I want to search, filter, sort, and export station and mission data.                               | Should   | 2        |
|    |                                     | 4.4   | As a supervisor, I want to consult historical data and trend visualizations.                                        | Should   | 3        |

*Continued on next page...*

| ID | Feature                         | US ID | User Story                                                                                                       | Priority | Complexity |
|----|---------------------------------|-------|------------------------------------------------------------------------------------------------------------------|----------|------------|
|    |                                 | 4.5   | As an administrator, I want to manage charging station records within the platform.                              | Must     | 2          |
| 5  | Alerts & Notifications          | 5.1   | As a supervisor, I want to receive real-time alerts when a station reports a critical event.                     | Must     | 3          |
|    |                                 | 5.2   | As a supervisor, I want to configure alerts for critical events (station failure, data anomaly, missed mission). | Should   | 2          |
|    |                                 | 5.3   | As a technician, I want to receive push notifications when a new mission is assigned to me.                      | Should   | 2          |
|    |                                 | 5.4   | As a supervisor, I want to configure notification thresholds and alert recipients.                               | Could    | 2          |
| 6  | Advanced Features & Scalability | 6.1   | As a supervisor, I want the system to automatically detect anomalies using AI-based analysis.                    | Could    | 5          |
|    |                                 | 6.2   | As a supervisor, I want predictive maintenance suggestions based on collected station data.                      | Could    | 5          |
|    |                                 | 6.3   | As an administrator, I want to manage multiple station fleets under a SaaS multi-tenant model.                   | Could    | 5          |
|    |                                 | 6.4   | As an administrator, I want a dedicated tenant management portal to provision and configure new tenants.         | Could    | 5          |

Tableau 2.1: Product Backlog

### 2.5.3 Sprint Planning

The project is organized into **6 sprints over 4 months**, structured around two releases: a first release delivering the **Minimum Viable Product (MVP)**, and a second release focused on

supervision, alerts, and advanced features.

| Release         | Sprint                                 | Period<br>(Days) | Features                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----------------|----------------------------------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Release 1 – MVP | Sprint 1<br>Auth & User<br>Foundations | 15               | <b>Authentication &amp; User Management</b><br>(7 user stories – 14 pts) <ul style="list-style-type: none"><li>• Secure login for technicians, supervisors and admin</li><li>• Password reset via secure process</li><li>• Role-based access control enforcement</li><li>• Create, modify and deactivate user accounts</li><li>• Manage role assignments and access permissions</li></ul>                                                                                                                 |
|                 | Sprint 2<br>Mission<br>Management      | 20               | <b>Mission Management (Web &amp; Mobile)</b> (9 user stories – 20 pts) <ul style="list-style-type: none"><li>• Mission creation, assignment, and scheduling</li><li>• Real-time lifecycle tracking (pending, in progress, completed, incident)</li><li>• Intervention history per station and per technician</li><li>• Technician mobile app: mission list and status updates</li><li>• Offline mode with automatic synchronization</li><li>• Optional geolocation capture at intervention time</li></ul> |

*Continued on next page...*

| Release                  | Sprint                                | Period<br>(Days) | Features                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|--------------------------|---------------------------------------|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Release 2 – Improvements | Sprint 3<br>Station &<br>Data         | 20               | <b>Station Monitoring &amp; Data Ingestion</b><br>(5 user stories – 13 pts) <ul style="list-style-type: none"> <li>• Automatic connection to EV stations via native protocols/APIs</li> <li>• Continuous real-time data ingestion (status, measurements, alerts)</li> <li>• Automatic data validation and anomaly handling</li> <li>• Normalization and structured storage</li> <li>• Collection logs and collector health monitoring</li> </ul> |
|                          | Sprint 4<br>Supervision<br>Dashboard  | 20               | <b>Supervision Dashboard</b> (5 user stories – 13 pts) <ul style="list-style-type: none"> <li>• Real-time overview of the entire EV station fleet</li> <li>• Detailed status per station (operational, faulty, offline, maintenance)</li> <li>• Search, filter, sort, and export station and mission data</li> <li>• Historical data consultation and trend visualization</li> <li>• Station record management from the dashboard</li> </ul>     |
|                          | Sprint 5<br>Alerts &<br>Notifications | 20               | <b>Alerts &amp; Notifications</b> (4 user stories – 10 pts) <ul style="list-style-type: none"> <li>• Real-time alerts for critical station events</li> <li>• Configurable alert rules and thresholds</li> <li>• Push notifications for technicians on new mission assignment</li> <li>• Configurable notification recipients</li> </ul>                                                                                                          |

*Continued on next page...*

| Release                                                | Sprint                           | Period<br>(Days) | Features                                                                                                                                                                                                                                                                                                                                                                       |
|--------------------------------------------------------|----------------------------------|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                        | Sprint 6<br>Advanced<br>Features | 25               | <b>Advanced Features &amp; Scalability</b> (4 user stories – 20 pts) <ul style="list-style-type: none"><li>• AI-based anomaly detection on collected station data</li><li>• Predictive maintenance suggestions based on historical data</li><li>• Multi-fleet / multi-tenant SaaS architecture</li><li>• Tenant management portal for provisioning and configuration</li></ul> |
| <b>Summary: 34 User Stories – 90 Points – 120 Days</b> |                                  |                  |                                                                                                                                                                                                                                                                                                                                                                                |

---

**Tableau 2.2:** Sprint Planning

---

## 2.6 Application Architecture

In this section, we will present the architecture. We are interested in the physical and software architecture of the application.

### 2.6.1 Physical Architecture

The physical architecture of the system follows an **N-Tier** model, which separates the system into distinct layers, each with a well-defined responsibility. This separation ensures scalability, maintainability, and clear boundaries between the different components of the platform. The physical architecture is organized into four tiers:

— **Presentation Layer:**

This tier encompasses all client-facing interfaces:

- **Mobile application** used by field technicians .
- **Web supervision dashboard** used by supervisors and administrators.

— **Application Layer:**

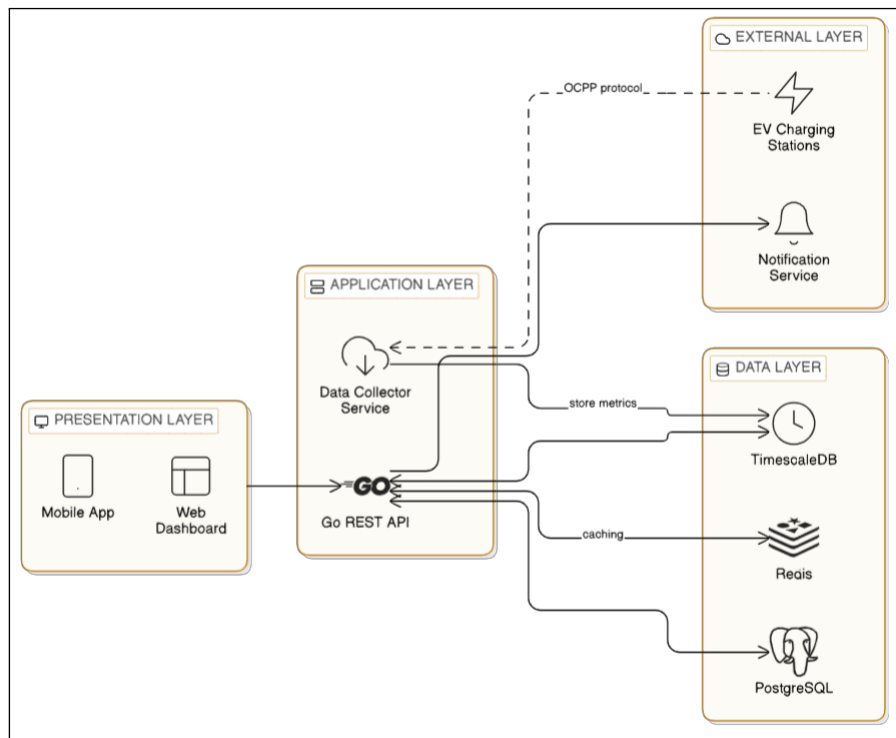
This tier hosts the core business logic of the system. It consists of the **Go REST API server**, which serves as the central entry point for all client requests, the **Data Collector Service**, which continuously ingests real-time data from EV charging stations.

— **Data Layer:**

This tier is responsible for data persistence and caching :

- **PostgreSQL** database: for relational data.
- **TimescaleDB** extension: for time-series storage of station metrics and historical measurements.
- **Redis** instance: for caching.
- **External Layer:**

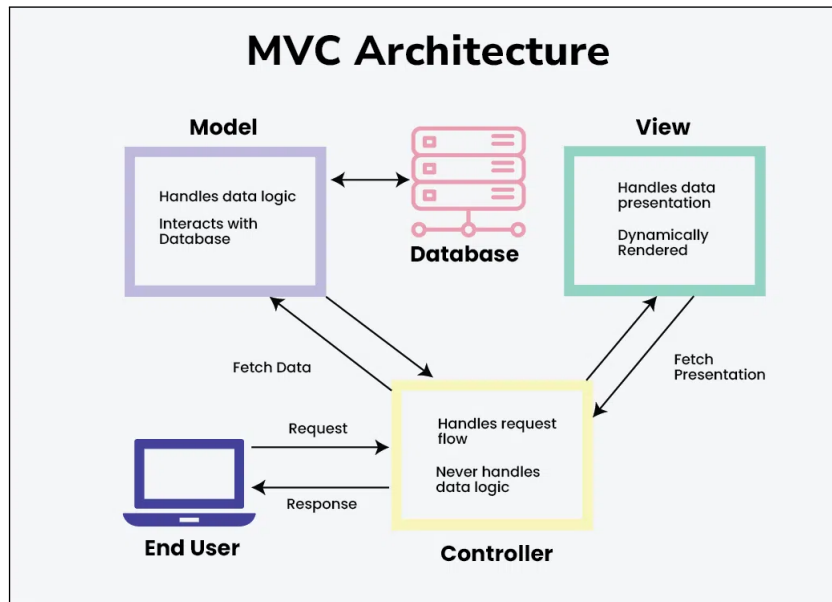
This tier represents the external systems that interact with the platform, primarily the **EV charging stations** communicating via the OCPP protocol, as well as any external notification.



**Figure 2.3:** Physical Architecture (N-Tier)

### 2.6.2 Logical Architecture

The logical architecture of the system follows the **MVC (Model-View-Controller)** pattern, which organizes the codebase into three distinct responsibilities and ensures a clean separation between data, business logic, and presentation.



**Figure 2.4:** Logical Architecture (MVC)

The MVC pattern is applied as follows:

- **Model:** The Model layer encapsulates the data structures and business rules of the system. It includes the core entities of the domain, along with their validation logic, relationships.
- **View:** The View layer corresponds to all client-facing interfaces that present data to the user and capture their interactions. It includes the **React-based web application**, the **Mobile application** for technicians. Views communicate exclusively through the Controller layer and never access the data layer directly.
- **Controller:** The Controller layer is implemented within the **Go REST API** and acts as the intermediary between Views and Models. It handles incoming HTTP and WebSocket requests, applies authentication and authorization middleware (JWT, role-based access, tenant identification), orchestrates business logic across services, and returns appropriate responses to the client.

## 2.7 Technical Choices and Development Environment

This section presents the technologies, frameworks, languages, and tools selected for the development of the system.

## 2.7.1 Programming Languages

### 2.7.1.1 Go (Golang)

Go is a statically typed, compiled programming language developed by Google in 2009 [4]. It was designed with simplicity, performance, and built-in concurrency in mind. Go's concurrency model is based on goroutines and channels, which allow thousands of lightweight threads to run simultaneously with minimal memory overhead. Its compilation speed, strong standard library, and native support for networking make it one of the most adopted languages for backend and systems programming.



**Figure 2.5:** Go Programming Language Logo

Figure 2.5 shows the logo of Go.

### 2.7.1.2 HTML

HTML (HyperText Markup Language) is the standard markup language for creating web pages [5]. It describes the structure of web content using a system of elements represented by tags. HTML5, the current standard maintained by the W3C, introduced semantic elements, multimedia support, and improved accessibility features that form the structural backbone of modern web applications.



**Figure 2.6:** HTML5 Logo

Figure 2.6 shows the logo of HTML5.

### 2.7.1.3 CSS

CSS (Cascading Style Sheets) is the language used to describe the presentation of a document written in HTML [6]. It controls layout, colors, typography, spacing, and responsiveness across different screen sizes. CSS3, the current version, introduced advanced features such as flexbox, grid layout, animations, and media queries, enabling the creation of fully responsive and visually

consistent web interfaces.



**Figure 2.7:** CSS3 Logo

Figure 2.7 shows the logo of CSS3.

#### **2.7.1.4 JavaScript**

JavaScript is a lightweight, interpreted scripting language standardized under the ECMAScript specification [7]. It is the only programming language natively supported by web browsers and serves as the runtime foundation for interactive web applications. With the introduction of Node.js, JavaScript also became widely used outside the browser, and it underlies both the React and React Native ecosystems used in this project.



**Figure 2.8:** JavaScript Logo

Figure 2.8 shows the logo of JavaScript.

#### **2.7.1.5 TypeScript**

TypeScript is an open-source programming language developed by Microsoft that extends JavaScript by adding optional static typing [8]. It compiles to plain JavaScript and is fully compatible with existing JavaScript codebases. TypeScript's type system enables early detection of errors at compile time, improves code readability, and provides powerful IDE support through autocompletion and inline documentation.



**Figure 2.9:** TypeScript Logo

Figure 2.9 shows the logo of TypeScript.

## 2.7.2 API Layer

### 2.7.2.1 GraphQL

GraphQL is an open-source query language for APIs and a server-side runtime for executing queries, originally developed by Meta in 2012 and publicly released in 2015 [9]. Unlike REST, GraphQL exposes a single typed endpoint through which clients can request precisely the fields they need, avoiding over-fetching and under-fetching. Its schema definition language (SDL) serves as a contract between the client and the server, and its subscription mechanism enables real-time data push over WebSocket connections.



**Figure 2.10:** GraphQL Logo

Figure 2.10 shows the logo of GraphQL.

## 2.7.3 Frameworks

### 2.7.3.1 Gin

Gin is an open-source HTTP web framework written in Go [10]. It is built on top of Go's native `net/http` package and offers a high-performance routing engine, middleware chaining, request binding, and JSON rendering. Gin is one of the most widely used Go web frameworks due to its minimalist design and benchmark performance that significantly outperforms comparable frameworks.



**Figure 2.11:** Gin Framework Logo

Figure 2.11 shows the logo of Gin.

### 2.7.3.2 gqlgen

`gqlgen` is a schema-first GraphQL server library for Go [11]. It generates type-safe Go code from a GraphQL schema definition, ensuring that the server implementation always conforms to the

API contract. gqlgen supports queries, mutations, and subscriptions out of the box, and integrates seamlessly with standard Go HTTP servers such as Gin.

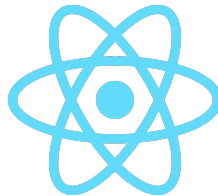


**Figure 2.12:** gqlgen Logo

Figure 2.12 shows the logo of gqlgen.

### **2.7.3.3 React**

React is an open-source JavaScript library developed and maintained by Meta for building user interfaces [12]. It introduced a component-based architecture and a virtual DOM that efficiently updates only the parts of the interface that have changed. React’s unidirectional data flow and rich ecosystem of complementary libraries make it the most widely adopted frontend framework for building complex, interactive web applications.



**Figure 2.13:** React Logo

Figure 2.13 shows the logo of React.

### **2.7.3.4 React Native**

React Native is an open-source mobile application framework created by Meta that enables building native iOS and Android applications using JavaScript and TypeScript [13]. It renders components using the platform’s native UI primitives rather than WebViews, delivering a genuine native experience while sharing business logic across platforms. Expo, a managed workflow built on top of React Native, further simplifies access to device APIs and over-the-air updates.



**Figure 2.14:** React Native Logo

Figure 2.14 shows the logo of React Native.

#### **2.7.3.5 Apollo Client**

Apollo Client is a comprehensive GraphQL client library for JavaScript and TypeScript [14]. It manages the full lifecycle of GraphQL operations on the client side, including query execution, mutation handling, subscription management, and normalized caching. Apollo Client integrates natively with both React and React Native, making it the standard choice for consuming GraphQL APIs in frontend applications.



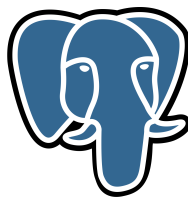
**Figure 2.15:** Apollo Client Logo

Figure 2.15 shows the logo of Apollo Client.

### **2.7.4 Database**

#### **2.7.4.1 PostgreSQL**

PostgreSQL is a powerful, open-source object-relational database management system with over 35 years of active development [15]. It is fully ACID-compliant and supports advanced features such as JSON columns, full-text search, foreign keys, triggers, and stored procedures. PostgreSQL is widely regarded as one of the most reliable and feature-rich open-source databases available.



**Figure 2.16:** PostgreSQL Logo

Figure 2.16 shows the logo of PostgreSQL.

#### **2.7.4.2 TimescaleDB**

TimescaleDB is an open-source time-series database built as an extension of PostgreSQL [16]. It introduces the concept of hypertables, which automatically partition time-series data across time and space for efficient ingestion and querying. TimescaleDB retains full SQL compatibility, making it a natural complement to PostgreSQL in architectures that handle both relational and

time-series data.



**Figure 2.17:** TimescaleDB Logo

Figure 2.17 shows the logo of TimescaleDB.

#### **2.7.4.3 Redis**

Redis (Remote Dictionary Server) is an open-source, in-memory data structure store that can be used as a database, cache, and message broker [17]. It supports a rich set of data structures including strings, hashes, lists, sets, and sorted sets. Redis's pub/sub messaging system enables low-latency event propagation between decoupled services, making it a standard component in real-time architectures.



**Figure 2.18:** Redis Logo

Figure 2.18 shows the logo of Redis.

### **2.7.5 Observability**

#### **2.7.5.1 Elasticsearch**

Elasticsearch is a distributed, RESTful search and analytics engine built on Apache Lucene [18]. It is designed to store, search, and analyze large volumes of data in near real time. In the ELK Stack, Elasticsearch serves as the central index where all processed logs are stored and made searchable through its powerful full-text and structured query API.



**Figure 2.19:** Elasticsearch Logo

Figure 2.19 shows the logo of Elasticsearch.

### 2.7.5.2 Logstash

Logstash is an open-source data processing pipeline that ingests data from multiple sources, transforms it, and sends it to a desired destination [19]. In the ELK Stack, Logstash collects logs from the various system services, applies parsing and enrichment rules, and forwards the structured output to Elasticsearch for indexing.



**Figure 2.20:** Logstash Logo

Figure 2.20 shows the logo of Logstash.

### 2.7.5.3 Kibana

Kibana is an open-source data visualization and exploration tool designed to work with Elasticsearch [20]. It provides a web interface for searching, filtering, and visualizing log data through dashboards, charts, and maps. Kibana is the visual layer of the ELK Stack and enables teams to gain operational insights from centralized log data.



**Figure 2.21:** Kibana Logo

Figure 2.21 shows the logo of Kibana.

### 2.7.5.4 Prometheus

Prometheus is an open-source systems monitoring and alerting toolkit originally developed at SoundCloud [21]. It collects and stores metrics as time-series data, identified by metric name and key-value labels. Prometheus uses a pull-based scraping model where it periodically queries instrumented services for their current metric values, making it highly reliable and independent of the monitored application's availability.



**Figure 2.22:** Prometheus Logo

Figure 2.22 shows the logo of Prometheus.

#### **2.7.5.5 Grafana**

Grafana is an open-source analytics and interactive visualization platform [22]. It connects to multiple data sources simultaneously and allows users to build rich dashboards combining metrics, logs, and traces in a single interface. Grafana also provides a built-in alerting engine capable of triggering notifications through various channels when defined thresholds are exceeded.



**Figure 2.23:** Grafana Logo

Figure 2.23 shows the logo of Grafana.

### **2.7.6 Development Tools**

#### **2.7.6.1 GoLand**

GoLand is a cross-platform IDE developed by JetBrains specifically for Go development [23]. It provides deep language-aware code completion, refactoring, inline documentation, an integrated debugger, and a built-in test runner. GoLand also offers native support for Docker, databases, and version control systems, making it a comprehensive development environment for Go projects.

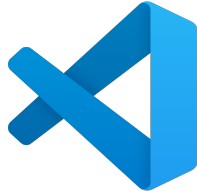


**Figure 2.24:** GoLand IDE Logo

Figure 2.24 shows the logo of GoLand.

### 2.7.6.2 Visual Studio Code

Visual Studio Code is a free, open-source code editor developed by Microsoft [24]. It supports a wide range of programming languages and offers a rich extension marketplace covering TypeScript, React, React Native, GraphQL, and Expo. Its lightweight architecture combined with powerful features such as IntelliSense, integrated terminal, and Git support make it the standard editor for frontend and mobile development.



**Figure 2.25:** Visual Studio Code Logo

Figure 2.25 shows the logo of Visual Studio Code.

### 2.7.6.3 Postman

Postman is a collaboration platform for API development [25]. It provides a graphical interface for designing, testing, and documenting APIs. Postman supports GraphQL natively, allowing developers to send queries and mutations, inspect responses, and validate schema behavior without writing client code.



**Figure 2.26:** Postman Logo

Figure 2.26 shows the logo of Postman.

### 2.7.6.4 Docker

Docker is an open-source platform for developing, shipping, and running applications in containers [26]. Containers package an application and all its dependencies into a single portable unit, ensuring consistent behavior across different environments. Docker Compose extends this by allowing the definition and orchestration of multi-container applications through a single configuration file.



**Figure 2.27:** Docker Logo

Figure 2.27 shows the logo of Docker.

#### **2.7.6.5 Git**

Git is a free and open-source distributed version control system designed to handle projects of any size with speed and efficiency [27]. It allows multiple developers to work on the same codebase simultaneously through branching and merging, and maintains a complete history of all changes made to the project.

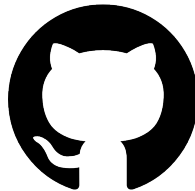


**Figure 2.28:** Git Logo

Figure 2.28 shows the logo of Git.

#### **2.7.6.6 GitHub**

GitHub is a web-based hosting platform for Git repositories [28]. It provides tools for code review, issue tracking, project management, and continuous integration through GitHub Actions. GitHub is the most widely used platform for open-source collaboration and serves as the central remote repository for this project.



**Figure 2.29:** GitHub Logo

Figure 2.29 shows the logo of GitHub.

## 2.8 Conclusion

In this chapter we presented the conceptual study, divided into two parts. The first covers the technical design, while the second outlines the technical choices and work environment. The description of the first release is addressed in the next chapter.

Also, we identified the system stakeholders, the functional and non-functional requirements, the product backlog, and the sprint planning in the first part, while the second part details the technical choices and work environment. The following chapters explain each release in depth.

---

# RELEASE 1: MVP

---

## Plan

|   |                                                           |    |
|---|-----------------------------------------------------------|----|
| 1 | Introduction . . . . .                                    | 38 |
| 2 | Release 1 Backlog . . . . .                               | 38 |
| 3 | Refined Use Case Diagram for Release 1 . . . . .          | 40 |
| 4 | Class Diagram of Release 1 . . . . .                      | 40 |
| 5 | Sprint 1: Authentication and User Foundations . . . . .   | 41 |
| 6 | Sprint 2: Mission Management . . . . .                    | 48 |
| 7 | Sprint 3: Station Monitoring and Data Ingestion . . . . . | 54 |
| 8 | Conclusion . . . . .                                      | 60 |

### 3.1 Introduction

In this chapter, we will present the different stages of the first release: Sprint 1 covering authentication and user foundations, Sprint 2 dedicated to mission management across both the web and mobile interfaces, and Sprint 3 focusing on station monitoring and automated data ingestion.

### 3.2 Release 1 Backlog

| ID | Feature                                 | US ID | User Story                                                                                                               | Priority | Complexity |
|----|-----------------------------------------|-------|--------------------------------------------------------------------------------------------------------------------------|----------|------------|
| 1  | Authentication<br>& User<br>Foundations | 1.1   | As a user, I want to log in securely to the system.                                                                      | Must     | 1          |
|    |                                         | 1.2   | As a user, I want to reset my password via a secure process.                                                             | Should   | 2          |
|    |                                         | 1.3   | As an administrator, I want role-based access control enforced across all components.                                    | Must     | 2          |
|    |                                         | 1.4   | As an administrator, I want to create technician and supervisor accounts.                                                | Must     | 2          |
|    |                                         | 1.5   | As an administrator, I want to modify and deactivate user accounts.                                                      | Must     | 2          |
|    |                                         | 1.6   | As an administrator, I want to manage role assignments and access permissions.                                           | Must     | 2          |
|    |                                         | 1.7   | As an administrator, I want to consult system usage statistics and audit logs.                                           | Should   | 3          |
| 2  | Mission<br>Management<br>(Web & Mobile) | 2.1   | As a supervisor, I want to create and assign intervention missions to technicians.                                       | Must     | 3          |
|    |                                         | 2.2   | As a supervisor, I want to track the lifecycle of each mission in real time (pending, in progress, completed, incident). | Must     | 3          |

*Continued on next page...*

| ID | Feature                             | US ID | User Story                                                                                                                          | Priority | Complexi |
|----|-------------------------------------|-------|-------------------------------------------------------------------------------------------------------------------------------------|----------|----------|
|    |                                     | 2.3   | As a supervisor, I want to consult the full intervention history per station and per technician.                                    | Should   | 2        |
|    |                                     | 2.4   | As an administrator, I want to manage mission types and assignment rules.                                                           | Could    | 2        |
|    |                                     | 2.5   | As a technician, I want to view the list of missions assigned to me with their details (location, description, priority, deadline). | Must     | 2        |
|    |                                     | 2.6   | As a technician, I want to update my mission status in real time with minimal interaction.                                          | Must     | 2        |
|    |                                     | 2.7   | As a technician, I want to use the application in offline mode with automatic synchronization upon reconnection.                    | Must     | 3        |
|    |                                     | 2.8   | As a technician, I want to consult my personal intervention history.                                                                | Should   | 1        |
|    |                                     | 2.9   | As a technician, I want my geolocation to be optionally captured at the time of intervention.                                       | Could    | 2        |
| 3  | Station Monitoring & Data Ingestion | 3.1   | As a supervisor, I want the system to automatically connect to EV charging stations and retrieve their data.                        | Must     | 3        |
|    |                                     | 3.2   | As a supervisor, I want station data (status, measurements, alerts) to be ingested continuously in real time.                       | Must     | 3        |
|    |                                     | 3.3   | As a supervisor, I want incoming data to be validated and anomalies handled automatically.                                          | Must     | 2        |

*Continued on next page...*

| ID | Feature | US ID | User Story                                                                                          | Priority | Complexi |
|----|---------|-------|-----------------------------------------------------------------------------------------------------|----------|----------|
|    |         | 3.4   | As a supervisor, I want collected data to be normalized and stored in a structured, unified format. | Must     | 2        |
|    |         | 3.5   | As an administrator, I want to consult collection logs and monitor the collector's health.          | Should   | 2        |

**Tableau 3.1:** Release 1 Backlog (MVP)

### 3.3 Refined Use Case Diagram for Release 1

Figure 3.1 shows the refined use case diagram for Release 1, covering authentication and user management, mission management across web and mobile, and automated station data ingestion.



**Figure 3.1:** Refined Use Case Diagram – Release 1

### 3.4 Class Diagram of Release 1

Figure 3.2 shows the class diagram of Release 1, illustrating the main entities: User, Role, Mission, Station, Collector, and Log, along with their attributes, methods, and relationships.



**Figure 3.2:** Class Diagram – Release 1

## 3.5 Sprint 1: Authentication and User Foundations

### 3.5.1 Sprint 1 Objectives

The objective of Sprint 1 is to establish the security foundation of the platform. This sprint delivers the authentication system allowing technicians, supervisors, and administrators to log in securely, as well as the user management module enabling the administrator to create, modify, and deactivate accounts and manage role assignments across the platform.

### 3.5.2 Refined Use Case Diagram for Sprint 1

Figure 3.3 shows the refined use case diagram for Sprint 1, detailing the interactions between the Administrator, Supervisor, and Technician actors with the authentication and user management functionalities.



**Figure 3.3:** Refined Use Case Diagram – Sprint 1

### 3.5.3 Analysis and Design for Sprint 1

#### 3.5.3.1 Use Case «Authenticate»

Textual description of the use case «Authenticate»

|                             |                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case</b>             | Authenticate                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Actor</b>                | Administrator, Supervisor, Technician                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Objective</b>            | Allow system users to log in securely and access their respective workspaces according to their assigned role.                                                                                                                                                                                                                                                                                                          |
| <b>Precondition</b>         | The user must have a valid account with an email and password registered in the system.                                                                                                                                                                                                                                                                                                                                 |
| <b>Post-condition</b>       | Upon successful authentication, the user is redirected to their role-specific home interface and a JWT token is issued.                                                                                                                                                                                                                                                                                                 |
| <b>Nominal Scenario</b>     | <ol style="list-style-type: none"><li>1. The user launches the application.</li><li>2. The system displays the authentication page.</li><li>3. The user enters their credentials (email and password).</li><li>4. The system validates the provided credentials.</li><li>5. The system generates a JWT token and establishes the session.</li><li>6. The user is redirected to their role-specific dashboard.</li></ol> |
| <b>Alternative Scenario</b> | <p><b>A1</b> The entered credentials are incorrect.</p> <p><b>A2</b> The system displays an error message on the screen.</p> <p><b>A3</b> The system returns to step 2 of the nominal scenario until authentication succeeds or the session is abandoned.</p>                                                                                                                                                           |

**Tableau 3.2:** Textual Description – Use Case «Authenticate»

### Sequence Diagram – Use Case «Login»

Figure 3.4 shows the sequence diagram of the Login use case, illustrating the interactions between the user, the GraphQL API, the authentication service, and the database.



**Figure 3.4:** Sequence Diagram – Login

#### 3.5.3.2 Use Case «User Management»

Textual description of the use case «User Management»

|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case</b>             | User Management                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Actor</b>                | Administrator                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Objective</b>            | Allow the administrator to create, modify, deactivate, and manage role assignments for all user accounts on the platform.                                                                                                                                                                                                                                                                                                                                    |
| <b>Precondition</b>         | The administrator must be authenticated and have full administrative privileges.                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Post-condition</b>       | The user account is successfully created, updated, or deactivated and the changes are persisted in the database.                                                                                                                                                                                                                                                                                                                                             |
| <b>Nominal Scenario</b>     | <ol style="list-style-type: none"><li>1. The administrator accesses the user management interface.</li><li>2. The system displays the list of existing user accounts.</li><li>3. The administrator selects the desired action (create, modify, or deactivate).</li><li>4. The administrator fills in the required information.</li><li>5. The system validates the input data.</li><li>6. The system saves the changes and confirms the operation.</li></ol> |
| <b>Alternative Scenario</b> | <p><b>A1</b> The provided data is invalid or incomplete.</p> <p><b>A2</b> The system displays a validation error message.</p> <p><b>A3</b> The system returns to step 4 of the nominal scenario until valid data is submitted.</p>                                                                                                                                                                                                                           |

**Tableau 3.3:** Textual Description – Use Case «User Management»

### Sequence Diagram – Use Case «Add User»

Figure 3.5 shows the sequence diagram of the Add User use case, illustrating the interactions between the administrator, the GraphQL API, and the database during the account creation

process.



**Figure 3.5:** Sequence Diagram – Add User

#### **3.5.4 Realization of Sprint 1**

Figure 3.6 and Figure 3.7 show the implemented interfaces for Sprint 1, covering the authentication screen and the user management dashboard.



**Figure 3.6:** Sprint 1 – Login Interface



**Figure 3.7:** Sprint 1 – User Management Interface

## 3.6 Sprint 2: Mission Management

### 3.6.1 Sprint 2 Objectives

The objective of Sprint 2 is to deliver the full mission management feature across both the web interface for supervisors and the mobile application for field technicians. Supervisors will be able to create, assign, and track intervention missions in real time, while technicians will be able to consult their assigned missions, update their status, and operate in offline mode with automatic synchronization upon reconnection.

### 3.6.2 Refined Use Case Diagram for Sprint 2

Figure 3.8 shows the refined use case diagram for Sprint 2, detailing the interactions between the Supervisor and Technician actors with the mission management functionalities on both web and mobile.



**Figure 3.8:** Refined Use Case Diagram – Sprint 2

### 3.6.3 Analysis and Design for Sprint 2

#### 3.6.3.1 Use Case «Create and Assign Mission»

Textual description of the use case «Create and Assign Mission»

|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case</b>             | Create and Assign Mission                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Actor</b>                | Supervisor                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Objective</b>            | Allow supervisors to create intervention missions and assign them to field technicians for a specific EV charging station.                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Precondition</b>         | The supervisor must be authenticated. At least one technician account and one charging station must exist in the system.                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Post-condition</b>       | The mission is successfully created, assigned to a technician, persisted in the database, and the technician is notified.                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Nominal Scenario</b>     | <ol style="list-style-type: none"> <li>1. The supervisor accesses the mission management interface.</li> <li>2. The system displays the list of existing missions.</li> <li>3. The supervisor creates a new mission by specifying the station, description, priority, and deadline.</li> <li>4. The supervisor assigns the mission to a technician.</li> <li>5. The system saves the mission and notifies the assigned technician.</li> <li>6. The supervisor monitors the mission lifecycle in real time.</li> </ol> |
| <b>Alternative Scenario</b> | <p><b>A1</b> No technician is available for assignment.</p> <p><b>A2</b> The system displays a warning and saves the mission as unassigned.</p> <p><b>A3</b> The supervisor can assign the mission later from the mission detail view.</p>                                                                                                                                                                                                                                                                            |

**Tableau 3.4:** Textual Description – Use Case «Create and Assign Mission»

#### Sequence Diagram – Use Case «Create and Assign Mission»

Figure 3.9 shows the sequence diagram of the Create and Assign Mission use case, illustrating

the interactions between the supervisor, the GraphQL API, the notification service, and the database.



**Figure 3.9:** Sequence Diagram – Create and Assign Mission

#### 3.6.3.2 Use Case «Update Mission Status»

Textual description of the use case «Update Mission Status»

|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case</b>             | Update Mission Status                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Actor</b>                | Technician                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Objective</b>            | Allow the technician to update the status of an assigned mission directly from the mobile application in real time.                                                                                                                                                                                                                                                                                                                                                              |
| <b>Precondition</b>         | The technician must be authenticated and have at least one mission assigned to them.                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Post-condition</b>       | The mission status is updated in the system and the supervisor is notified of the change in real time.                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Nominal Scenario</b>     | <ol style="list-style-type: none"><li>1. The technician opens the mobile application.</li><li>2. The system displays the list of assigned missions.</li><li>3. The technician selects a mission and views its details.</li><li>4. The technician updates the mission status (In Progress, Completed, or Incident).</li><li>5. The system saves the update and synchronizes with the server.</li><li>6. The supervisor receives a real-time notification of the change.</li></ol> |
| <b>Alternative Scenario</b> | <p><b>A1</b> The technician has no internet connection.</p> <p><b>A2</b> The update is stored locally in offline mode.</p> <p><b>A3</b> Upon reconnection, the system automatically synchronizes the pending update with the server.</p>                                                                                                                                                                                                                                         |

**Tableau 3.5:** Textual Description – Use Case «Update Mission Status»**Sequence Diagram – Use Case «Update Mission Status»**

Figure 3.10 shows the sequence diagram of the Update Mission Status use case, including the offline synchronization scenario.



**Figure 3.10:** Sequence Diagram – Update Mission Status

### 3.6.4 Realization of Sprint 2

Figure 3.11 and Figure 3.12 show the implemented interfaces for Sprint 2, covering the mission management dashboard and the technician mobile application.



**Figure 3.11:** Sprint 2 – Mission Management Interface (Web)



**Figure 3.12:** Sprint 2 – Technician Mobile Application

## 3.7 Sprint 3: Station Monitoring and Data Ingestion

### 3.7.1 Sprint 3 Objectives

The objective of Sprint 3 is to implement the automated data collector service and the station monitoring foundation. The collector connects securely to EV charging stations, ingests their data continuously in real time, validates and normalizes the collected measurements, and stores them in the TimescaleDB time-series database. The administrator can also consult collector health logs and monitor ingestion status.

### 3.7.2 Refined Use Case Diagram for Sprint 3

Figure 3.13 shows the refined use case diagram for Sprint 3, detailing the interactions between the Data Collector, the Administrator, and the EV Charging Station with the data ingestion and monitoring functionalities.



**Figure 3.13:** Refined Use Case Diagram – Sprint 3

### 3.7.3 Analysis and Design for Sprint 3

#### 3.7.3.1 Use Case «Automated Data Collection»

Textual description of the use case «Automated Data Collection»

|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case</b>             | Automated Data Collection                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Actor</b>                | Data Collector (System)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Objective</b>            | Automatically connect to EV charging stations, retrieve their real-time data, validate and normalize it, and store it in a structured format for supervision and analysis.                                                                                                                                                                                                                                                                                                                                 |
| <b>Precondition</b>         | The collector service is running and the EV charging stations are reachable via their native protocol.                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Post-condition</b>       | The collected data is validated, normalized, and persisted in TimescaleDB and published to Redis for real-time propagation.                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Nominal Scenario</b>     | <ol style="list-style-type: none"><li>1. The collector service starts and reads the list of configured stations.</li><li>2. The collector establishes a secure connection to each station.</li><li>3. The station returns its current data (status, measurements, signals).</li><li>4. The collector validates the received data against defined rules.</li><li>5. The collector normalizes the data into a unified format.</li><li>6. The data is stored in TimescaleDB and published to Redis.</li></ol> |
| <b>Alternative Scenario</b> | <p><b>A1</b> The collector cannot reach a station.</p> <p><b>A2</b> The system logs the failure and marks the station as offline.</p> <p><b>A3</b> The collector retries according to a configured retry policy.</p>                                                                                                                                                                                                                                                                                       |

**Tableau 3.6:** Textual Description – Use Case «Automated Data Collection»

**Sequence Diagram – Use Case «Automated Data Collection»**

Figure 3.14 shows the sequence diagram of the Automated Data Collection use case, illustrating the interactions between the collector service, the EV charging station, the validation layer, TimescaleDB, and Redis.



**Figure 3.14:** Sequence Diagram – Automated Data Collection

**3.7.3.2 Use Case «Monitor Collector Health»**

**Textual description of the use case «Monitor Collector Health»**

|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case</b>             | Monitor Collector Health                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Actor</b>                | Administrator                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Objective</b>            | Allow the administrator to consult collection logs, monitor the health status of the collector service, and identify any ingestion failures or connection issues.                                                                                                                                                                                                                                                                                                    |
| <b>Precondition</b>         | The administrator must be authenticated with administrative privileges.                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Post-condition</b>       | The administrator has a clear view of the collector's operational status and can take corrective action if needed.                                                                                                                                                                                                                                                                                                                                                   |
| <b>Nominal Scenario</b>     | <ol style="list-style-type: none"><li>1. The administrator accesses the collector monitoring interface.</li><li>2. The system displays the current collector status (active, degraded, offline).</li><li>3. The administrator consults the ingestion logs for each station.</li><li>4. The administrator identifies any failed connections or validation errors.</li><li>5. The administrator takes corrective action or escalates the issue if necessary.</li></ol> |
| <b>Alternative Scenario</b> | <p><b>A1</b> The collector service is completely offline.</p> <p><b>A2</b> The system displays a critical alert on the admin dashboard.</p> <p><b>A3</b> The administrator restarts the collector service from the management interface.</p>                                                                                                                                                                                                                         |

**Tableau 3.7:** Textual Description – Use Case «Monitor Collector Health»**Sequence Diagram – Use Case «Monitor Collector Health»**

Figure 3.15 shows the sequence diagram of the Monitor Collector Health use case, illustrating the interactions between the administrator, the GraphQL API, and the logging and monitoring

layer.



**Figure 3.15:** Sequence Diagram – Monitor Collector Health

### 3.7.4 Realization of Sprint 3

Figure 3.16 and Figure 3.17 show the implemented interfaces for Sprint 3, covering the data collector monitoring dashboard and the ingestion logs interface.



**Figure 3.16:** Sprint 3 – Data Collector Monitoring Interface



**Figure 3.17:** Sprint 3 – Ingestion Logs Interface

## 3.8 Conclusion

This chapter presented the complete realization of Release 1. Sprint 1 established the security foundation through authentication and user management. Sprint 2 delivered the full mission management feature across both the web supervisor interface and the technician mobile application, including offline mode support. Sprint 3 implemented the automated data collector service, enabling continuous real-time ingestion of EV station measurements into TimescaleDB. The following chapter will present the realization of Release 2, which introduces the supervision dashboard, the alert and notification system, and the platform's advanced features.

---

# RELEASE 2: IMPROVEMENTS

---

## Plan

|   |                                                       |    |
|---|-------------------------------------------------------|----|
| 1 | Introduction . . . . .                                | 62 |
| 2 | Release 2 Backlog . . . . .                           | 62 |
| 3 | Refined Use Case Diagram for Release 2 . . . . .      | 63 |
| 4 | Class Diagram of Release 2 . . . . .                  | 64 |
| 5 | Sprint 4: Supervision Dashboard . . . . .             | 65 |
| 6 | Sprint 5: Alerts & Notifications . . . . .            | 72 |
| 7 | Sprint 6: Advanced Features and Scalability . . . . . | 78 |
| 8 | Conclusion . . . . .                                  | 84 |

## 4.1 Introduction

In this chapter, we will present the different stages of the second release: Sprint 4 delivering the web supervision dashboard, Sprint 5 introducing the alert and notification system, and Sprint 6 covering the advanced features including AI-based anomaly detection and multi-tenant SaaS architecture.

## 4.2 Release 2 Backlog

| ID | Feature                | US ID | User Story                                                                                                          | Priority | Complexi |
|----|------------------------|-------|---------------------------------------------------------------------------------------------------------------------|----------|----------|
| 4  | Supervision Dashboard  | 4.1   | As a supervisor, I want to access a real-time dashboard showing the status of all EV charging stations.             | Must     | 3        |
|    |                        | 4.2   | As a supervisor, I want to view detailed information per station (operational, faulty, offline, under maintenance). | Must     | 2        |
|    |                        | 4.3   | As a supervisor, I want to search, filter, sort, and export station and mission data.                               | Should   | 2        |
|    |                        | 4.4   | As a supervisor, I want to consult historical data and trend visualizations.                                        | Should   | 3        |
|    |                        | 4.5   | As an administrator, I want to manage charging station records within the platform.                                 | Must     | 2        |
| 5  | Alerts & Notifications | 5.1   | As a supervisor, I want to receive real-time alerts when a station reports a critical event.                        | Must     | 3        |
|    |                        | 5.2   | As a supervisor, I want to configure alerts for critical events (station failure, data anomaly, missed mission).    | Should   | 2        |
|    |                        | 5.3   | As a technician, I want to receive push notifications when a new mission is assigned to me.                         | Should   | 2        |

*Continued on next page...*

| ID | Feature                         | US ID | User Story                                                                                               | Priority | Complexi |
|----|---------------------------------|-------|----------------------------------------------------------------------------------------------------------|----------|----------|
|    |                                 | 5.4   | As a supervisor, I want to configure notification thresholds and alert recipients.                       | Could    | 2        |
| 6  | Advanced Features & Scalability | 6.1   | As a supervisor, I want the system to automatically detect anomalies using AI-based analysis.            | Could    | 5        |
|    |                                 | 6.2   | As a supervisor, I want predictive maintenance suggestions based on collected station data.              | Could    | 5        |
|    |                                 | 6.3   | As an administrator, I want to manage multiple station fleets under a SaaS multi-tenant model.           | Could    | 5        |
|    |                                 | 6.4   | As an administrator, I want a dedicated tenant management portal to provision and configure new tenants. | Could    | 5        |

**Tableau 4.1:** Release 2 Backlog (Improvements)

### 4.3 Refined Use Case Diagram for Release 2

Figure 4.1 shows the refined use case diagram for Release 2, covering the supervision dashboard, the alert and notification system, and the advanced features including AI integration and multi-tenancy.



**Figure 4.1:** Refined Use Case Diagram – Release 2

## 4.4 Class Diagram of Release 2

Figure 4.2 shows the class diagram of Release 2, extending the Release 1 model with the entities introduced in this release: Alert, Notification, Report, and Tenant.



**Figure 4.2:** Class Diagram – Release 2

## 4.5 Sprint 4: Supervision Dashboard

### 4.5.1 Sprint 4 Objectives

The objective of Sprint 4 is to deliver the web supervision dashboard, providing supervisors with a real-time, centralized view of the entire EV station fleet. This sprint also introduces station record management for administrators and historical data visualization through trend charts.

### 4.5.2 Refined Use Case Diagram for Sprint 4

Figure 4.3 shows the refined use case diagram for Sprint 4, detailing the interactions between the Supervisor and Administrator actors with the supervision dashboard functionalities.



**Figure 4.3:** Refined Use Case Diagram – Sprint 4

### 4.5.3 Analysis and Design for Sprint 4

#### 4.5.3.1 Use Case «Monitor Station Fleet»

Textual description of the use case «Monitor Station Fleet»

|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case</b>             | Monitor Station Fleet                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Actor</b>                | Supervisor                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Objective</b>            | Allow supervisors to monitor the entire EV charging station fleet in real time through a centralized web dashboard.                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Precondition</b>         | The supervisor must be authenticated. The data collector must be active and ingesting station data.                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Post-condition</b>       | The supervisor has an up-to-date view of all station statuses and can take informed operational decisions.                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Nominal Scenario</b>     | <ol style="list-style-type: none"> <li>1. The supervisor logs in and accesses the supervision dashboard.</li> <li>2. The system fetches the latest station data via GraphQL.</li> <li>3. The dashboard displays the global fleet status with per-station indicators (operational, faulty, offline, maintenance).</li> <li>4. The supervisor selects a station to view its detailed information.</li> <li>5. The supervisor consults historical trends for the selected station.</li> <li>6. The supervisor exports the filtered data if needed.</li> </ol> |
| <b>Alternative Scenario</b> | <p><b>A1</b> The data collector is offline and no recent data is available.</p> <p><b>A2</b> The system displays a warning banner with the last successful sync timestamp.</p> <p><b>A3</b> The supervisor is redirected to the collector health monitoring view.</p>                                                                                                                                                                                                                                                                                      |

**Tableau 4.2:** Textual Description – Use Case «Monitor Station Fleet»**Sequence Diagram – Use Case «Monitor Station Fleet»**

Figure 4.4 shows the sequence diagram of the Monitor Station Fleet use case, illustrating the interactions between the supervisor, the React dashboard, the GraphQL API, Redis, and TimescaleDB.



**Figure 4.4:** Sequence Diagram – Monitor Station Fleet

#### 4.5.3.2 Use Case «Consult Historical Data»

Textual description of the use case «Consult Historical Data»

|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case</b>             | Consult Historical Data                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Actor</b>                | Supervisor                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Objective</b>            | Allow supervisors to consult historical station measurements and visualize trends over a selected time period.                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Precondition</b>         | The supervisor must be authenticated and historical data must be available in TimescaleDB.                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Post-condition</b>       | The supervisor views trend charts and can export the historical dataset for further analysis.                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Nominal Scenario</b>     | <ol style="list-style-type: none"> <li>1. The supervisor selects a station from the dashboard.</li> <li>2. The supervisor accesses the historical data view.</li> <li>3. The supervisor selects a time range (last 24h, 7 days, 30 days).</li> <li>4. The system queries TimescaleDB and returns the aggregated metrics.</li> <li>5. The dashboard renders trend charts for the selected period.</li> <li>6. The supervisor optionally exports the data as a report.</li> </ol> |
| <b>Alternative Scenario</b> | <p><b>A1</b> No historical data is available for the selected period.</p> <p><b>A2</b> The system displays an empty state message with the earliest available data date.</p> <p><b>A3</b> The supervisor adjusts the time range and retries.</p>                                                                                                                                                                                                                                |

**Tableau 4.3:** Textual Description – Use Case «Consult Historical Data»**Sequence Diagram – Use Case «Consult Historical Data»**

Figure 4.5 shows the sequence diagram of the Consult Historical Data use case, illustrating the interactions between the supervisor, the React dashboard, the GraphQL API, and TimescaleDB.



**Figure 4.5:** Sequence Diagram – Consult Historical Data

#### 4.5.4 Realization of Sprint 4

Figure 4.6 and Figure 4.7 show the implemented interfaces for Sprint 4, covering the real-time supervision dashboard and the historical data visualization interface.



**Figure 4.6:** Sprint 4 – Web Supervision Dashboard



**Figure 4.7:** Sprint 4 – Historical Data Visualization

## 4.6 Sprint 5: Alerts & Notifications

### 4.6.1 Sprint 5 Objectives

The objective of Sprint 5 is to implement the real-time alert and notification system. Supervisors will receive instant alerts for critical station events, configure alert rules and thresholds, and manage notification recipients. Technicians will receive push notifications on their mobile devices upon new mission assignment.

### 4.6.2 Refined Use Case Diagram for Sprint 5

Figure 4.8 shows the refined use case diagram for Sprint 5, detailing the interactions between the Supervisor and Technician actors with the alert and notification functionalities.



**Figure 4.8:** Refined Use Case Diagram – Sprint 5

### 4.6.3 Analysis and Design for Sprint 5

#### 4.6.3.1 Use Case «Receive Real-Time Alert»

Textual description of the use case «Receive Real-Time Alert»

|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case</b>             | Receive Real-Time Alert                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Actor</b>                | Supervisor                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Objective</b>            | Allow supervisors to receive instant alerts when a critical event is detected on a charging station, enabling rapid response.                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Precondition</b>         | The supervisor must be authenticated and connected to the dashboard. Alert rules must be configured.                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Post-condition</b>       | The supervisor is notified in real time and can acknowledge or escalate the alert.                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Nominal Scenario</b>     | <ol style="list-style-type: none"> <li>1. A critical event is detected (station failure, anomaly, overdue mission).</li> <li>2. The alert service generates an alert record and publishes it via Redis pub/sub.</li> <li>3. The GraphQL subscription propagates the alert to connected dashboard clients.</li> <li>4. The supervisor receives the alert in real time on the dashboard.</li> <li>5. The supervisor consults the alert details and takes action.</li> <li>6. The supervisor acknowledges or resolves the alert.</li> </ol> |
| <b>Alternative Scenario</b> | <p><b>A1</b> The supervisor is not connected at the time of the alert.</p> <p><b>A2</b> The alert is stored and displayed as unread on next login.</p> <p><b>A3</b> A push notification is also sent if configured.</p>                                                                                                                                                                                                                                                                                                                  |

**Tableau 4.4:** Textual Description – Use Case «Receive Real-Time Alert»**Sequence Diagram – Use Case «Receive Real-Time Alert»**

Figure 4.9 shows the sequence diagram of the Receive Real-Time Alert use case, illustrating the event propagation chain from the collector through Redis and the GraphQL subscription to the dashboard.



**Figure 4.9:** Sequence Diagram – Receive Real-Time Alert

#### **4.6.3.2 Use Case «Push Notification»**

**Textual description of the use case «Push Notification»**

|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case</b>             | Push Notification                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Actor</b>                | Technician                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Objective</b>            | Notify the technician via a push notification when a new mission is assigned to them.                                                                                                                                                                                                                                                                                                                                                    |
| <b>Precondition</b>         | The technician must have the mobile application installed and push notifications enabled on their device.                                                                                                                                                                                                                                                                                                                                |
| <b>Post-condition</b>       | The technician receives a push notification and is redirected to the mission detail screen.                                                                                                                                                                                                                                                                                                                                              |
| <b>Nominal Scenario</b>     | <ol style="list-style-type: none"> <li>1. A supervisor assigns a new mission to the technician.</li> <li>2. The system saves the mission and triggers the notification service.</li> <li>3. The notification service sends a push notification via FCM.</li> <li>4. The technician receives the notification on their mobile device.</li> <li>5. The technician taps the notification and accesses the mission detail screen.</li> </ol> |
| <b>Alternative Scenario</b> | <p><b>A1</b> The technician's device is offline.</p> <p><b>A2</b> The notification is queued by FCM and delivered upon reconnection.</p> <p><b>A3</b> The mission appears as unread on the technician's next application launch.</p>                                                                                                                                                                                                     |

**Tableau 4.5:** Textual Description – Use Case «Push Notification»**Sequence Diagram – Use Case «Push Notification»**

Figure 4.10 shows the sequence diagram of the Push Notification use case, illustrating the interactions between the supervisor, the GraphQL API, the notification service, FCM, and the technician's device.



**Figure 4.10:** Sequence Diagram – Push Notification

#### 4.6.4 Realization of Sprint 5

Figure 4.11 and Figure 4.12 show the implemented interfaces for Sprint 5, covering the alert management interface and the mobile push notification flow.



**Figure 4.11:** Sprint 5 – Alert Management Interface



**Figure 4.12:** Sprint 5 – Push Notification on Mobile

## 4.7 Sprint 6: Advanced Features and Scalability

### 4.7.1 Sprint 6 Objectives

The objective of Sprint 6 is to deliver the platform's advanced features: AI-based anomaly detection for proactive station monitoring, predictive maintenance suggestions based on historical time-series data, and the foundation of a multi-tenant SaaS architecture enabling the management of multiple independent station fleets under a single platform instance.

### 4.7.2 Refined Use Case Diagram for Sprint 6

Figure 4.13 shows the refined use case diagram for Sprint 6, detailing the interactions between the Supervisor, the Administrator, and the AI engine with the anomaly detection and multi-tenancy functionalities.



**Figure 4.13:** Refined Use Case Diagram – Sprint 6

### 4.7.3 Analysis and Design for Sprint 6

#### 4.7.3.1 Use Case «AI-Based Anomaly Detection»

Textual description of the use case «AI-Based Anomaly Detection»

|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case</b>             | AI-Based Anomaly Detection                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Actor</b>                | System (AI Engine), Supervisor                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Objective</b>            | Automatically detect abnormal patterns in collected station data and notify the supervisor before issues escalate into failures.                                                                                                                                                                                                                                                                                                                    |
| <b>Precondition</b>         | Sufficient historical station data must be available in TimescaleDB to run the anomaly detection model.                                                                                                                                                                                                                                                                                                                                             |
| <b>Post-condition</b>       | Detected anomalies are recorded as alerts and communicated to the supervisor in real time.                                                                                                                                                                                                                                                                                                                                                          |
| <b>Nominal Scenario</b>     | <ol style="list-style-type: none"> <li>1. The AI engine periodically queries TimescaleDB for recent metrics.</li> <li>2. The model identifies deviations from expected patterns.</li> <li>3. A detected anomaly is flagged and an alert record is created.</li> <li>4. The alert is propagated to the supervisor via GraphQL subscription.</li> <li>5. The supervisor reviews the anomaly and schedules a maintenance mission if needed.</li> </ol> |
| <b>Alternative Scenario</b> | <p><b>A1</b> The model confidence score is below the defined threshold.</p> <p><b>A2</b> The anomaly is flagged as a low-confidence warning.</p> <p><b>A3</b> The supervisor can confirm or dismiss the warning manually.</p>                                                                                                                                                                                                                       |

**Tableau 4.6:** Textual Description – Use Case «AI-Based Anomaly Detection»**Sequence Diagram – Use Case «AI-Based Anomaly Detection»**

Figure 4.14 shows the sequence diagram of the Anomaly Detection use case, illustrating the interactions between the AI engine, TimescaleDB, the alert service, Redis, and the supervisor's

dashboard.



**Figure 4.14:** Sequence Diagram – AI-Based Anomaly Detection

#### 4.7.3.2 Use Case «Provision New Tenant»

Textual description of the use case «Provision New Tenant»

|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case</b>             | Provision New Tenant                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Actor</b>                | Administrator                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Objective</b>            | Allow the platform administrator to create and provision a new independent tenant instance with its own isolated data, users, and station fleet under the shared SaaS infrastructure.                                                                                                                                                                                                                                                                   |
| <b>Precondition</b>         | The administrator must be authenticated with super-admin privileges.                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Post-condition</b>       | A new tenant is provisioned with its own PostgreSQL schema and a dedicated administrator account.                                                                                                                                                                                                                                                                                                                                                       |
| <b>Nominal Scenario</b>     | <ol style="list-style-type: none"><li>1. The super-admin accesses the tenant management portal.</li><li>2. The super-admin creates a new tenant with its name and configuration.</li><li>3. The system provisions a new PostgreSQL schema for the tenant.</li><li>4. The system creates a dedicated administrator account for the new tenant.</li><li>5. The tenant administrator logs in and configures their fleet and users independently.</li></ol> |
| <b>Alternative Scenario</b> | <p><b>A1</b> The tenant name already exists.</p> <p><b>A2</b> The system displays a conflict error.</p> <p><b>A3</b> The super-admin retries with a unique identifier.</p>                                                                                                                                                                                                                                                                              |

**Tableau 4.7:** Textual Description – Use Case «Provision New Tenant»**Sequence Diagram – Use Case «Provision New Tenant»**

Figure 4.15 shows the sequence diagram of the Provision New Tenant use case, illustrating the interactions between the super-admin, the GraphQL API, the tenant provisioning service, and the PostgreSQL database.



**Figure 4.15:** Sequence Diagram – Provision New Tenant

#### 4.7.4 Realization of Sprint 6

Figure 4.16 and Figure 4.17 show the implemented interfaces for Sprint 6, covering the AI anomaly detection dashboard and the tenant management portal.



**Figure 4.16:** Sprint 6 – AI Anomaly Detection Dashboard



**Figure 4.17:** Sprint 6 – Multi-Tenant Management Portal

## 4.8 Conclusion

This chapter presented the complete realization of Release 2. Sprint 4 delivered the web supervision dashboard with real-time station monitoring and historical data visualization. Sprint 5 introduced the alert and notification system, enabling real-time event propagation to supervisors and push notifications to field technicians. Sprint 6 completed the platform with AI-based anomaly detection, predictive maintenance suggestions, and the foundation of a multi-tenant SaaS architecture. Together, the two releases constitute a complete, scalable, and production-ready integrated system for EV charging station management and technician mission supervision.

# General Conclusion

This report presents our end-of-studies project carried out within the framework of our internship at **EntityService**, as part of our academic journey at **Tek-Up University** in computer engineering. This project offered us the opportunity to apply the theoretical and technical knowledge acquired throughout our studies, particularly in the development of modern, integrated, and real-time software systems.

Our project, entitled «*Conception et développement d’une plateforme de gestion et de supervision en temps réel des bornes de recharge pour véhicules électriques*», responds to a growing operational need: replacing a fragmented, spreadsheet-based workflow with a reliable, scalable, and professional platform capable of managing EV charging station data and technician missions in real time.

Through the development of this platform, we designed and implemented an integrated system composed of three core components: an automated data collector that continuously ingests real-time measurements from EV charging stations, a cross-platform mobile application dedicated to field technicians for managing their intervention missions, and a web supervision dashboard providing back-office teams with a comprehensive, real-time view of the entire station fleet.

Throughout this project, we adopted an agile development approach based on the **Scrum** methodology, enabling incremental and iterative delivery of the system across clearly defined sprints. This approach helped us structure our work into two releases and five sprints, each delivering functional and testable increments of the platform.

During our internship at EntityService, we successfully realized the following modules:

- Secure authentication and role-based user management for administrators, supervisors, and technicians;
- Mission management with real-time lifecycle tracking and assignment to field technicians;
- A cross-platform mobile application with offline mode and automatic synchronization;
- An automated data collector service for continuous EV station data ingestion, validation, and storage;
- A web supervision dashboard with real-time station monitoring, configurable alerts, and data visualization.

Thanks to these achievements, the platform enables EntityService’s client to significantly improve the supervision of their station fleet, reduce human error, gain operational traceability, and enhance the productivity of both field and back-office teams. Furthermore, the integration of a full observability stack combining **ELK**, **Prometheus**, and **Grafana** ensures efficient monitoring

and prepares the platform for future scaling.

However, this project represents a first version that opens the way to several possible evolutions. Among the planned improvements are the completion of the multi-tenant SaaS architecture enabling the management of multiple independent station fleets under a single platform instance, the integration of AI-based anomaly detection for proactive maintenance, and the addition of predictive maintenance suggestions based on historical time-series data collected by the platform.

In summary, this project constitutes a solid and innovative foundation for addressing the current challenges of EV charging station management. It delivers a reliable, scalable, and future-ready architecture, and offers a promising framework for further developments aimed at accompanying the digital transformation of the electric mobility sector.

# Webography

- [1] Object Management Group, « OMG Unified Modeling Language (UML), Version 2.5.1 », Object Management Group, Tech. Rep., 2017. [Online]. Available: <https://www.omg.org/spec/UML/2.5.1>
- [2] G. Booch, J. Rumbaugh, and I. Jacobson, *The Unified Modeling Language User Guide*, 2nd. Addison-Wesley, 2005.
- [3] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd. Addison-Wesley, 2003.
- [4] Google LLC, *The Go Programming Language*, 2009. [Online]. Available: <https://go.dev>
- [5] W3C, « HTML5 Specification », World Wide Web Consortium, Tech. Rep., 2014. [Online]. Available: <https://www.w3.org/TR/html5>
- [6] W3C, « CSS Snapshot », World Wide Web Consortium, Tech. Rep., 2023. [Online]. Available: <https://www.w3.org/TR/css-2023>
- [7] Ecma International, « ECMAScript 2023 Language Specification », Ecma International, Tech. Rep., 2023. [Online]. Available: <https://www.ecma-international.org/publications-and-standards/standards/ecma-262>
- [8] Microsoft, *TypeScript: JavaScript With Syntax For Types*, 2012. [Online]. Available: <https://www.typescriptlang.org>
- [9] GraphQL Foundation, *GraphQL Specification*, 2021. [Online]. Available: <https://spec.graphql.org>
- [10] Gin Contributors, *Gin Web Framework*, 2014. [Online]. Available: <https://gin-gonic.com>
- [11] 99designs, *gqlgen: A Go Library for Building GraphQL Servers*, 2018. [Online]. Available: <https://gqlgen.com>
- [12] Meta Platforms, *React: A JavaScript Library for Building User Interfaces*, 2013. [Online]. Available: <https://react.dev>
- [13] Meta Platforms, *React Native: Learn Once, Write Anywhere*, 2015. [Online]. Available: <https://reactnative.dev>
- [14] Apollo GraphQL, *Apollo Client Documentation*, 2016. [Online]. Available: <https://www.apollographql.com/docs/react>

- [15] PostgreSQL Global Development Group, *PostgreSQL: The World's Most Advanced Open Source Database*, 1996. [Online]. Available: <https://www.postgresql.org>
- [16] Timescale Inc., *TimescaleDB Documentation*, 2017. [Online]. Available: <https://docs.timescale.com>
- [17] Redis Ltd., *Redis Documentation*, 2009. [Online]. Available: <https://redis.io>
- [18] Elastic NV, *Elasticsearch: The Official Distributed Search and Analytics Engine*, 2010. [Online]. Available: <https://www.elastic.co/elasticsearch>
- [19] Elastic NV, *Logstash: Collect, Parse, Transform Logs*, 2010. [Online]. Available: <https://www.elastic.co/logstash>
- [20] Elastic NV, *Kibana: Your Window into the Elastic Stack*, 2013. [Online]. Available: <https://www.elastic.co/kibana>
- [21] Prometheus Authors, *Prometheus: Monitoring System and Time Series Database*, 2012. [Online]. Available: <https://prometheus.io>
- [22] Grafana Labs, *Grafana: The Open Observability Platform*, 2014. [Online]. Available: <https://grafana.com>
- [23] JetBrains, *GoLand: The Go IDE by JetBrains*, 2017. [Online]. Available: <https://www.jetbrains.com/go>
- [24] Microsoft, *Visual Studio Code*, 2015. [Online]. Available: <https://code.visualstudio.com>
- [25] Postman Inc., *Postman API Platform*, 2012. [Online]. Available: <https://www.postman.com>
- [26] Docker Inc., *Docker: Accelerated Container Application Development*, 2013. [Online]. Available: <https://www.docker.com>
- [27] Linus Torvalds, *Git: Distributed Version Control System*, 2005. [Online]. Available: <https://git-scm.com>
- [28] GitHub Inc., *GitHub: Where the World Builds Software*, 2008. [Online]. Available: <https://github.com>

# Résumé

Ce rapport présente le travail effectué dans le cadre du projet de fin d'études à Tek-Up University, réalisé au sein d'EntityService.

L'objectif est la conception et le développement d'une plateforme intégrée de gestion et supervision en temps réel des bornes de recharge pour véhicules électriques. La plateforme comprend trois composants : un collecteur automatique de données, une application mobile pour les techniciens de terrain, et un tableau de bord web de supervision.

Le système repose sur une architecture moderne combinant un backend en **Go** avec API **GraphQL**, un dashboard **React**, une application **React Native**, et une stack d'observabilité (**ELK**, **Prometheus**, **Grafana**). Le projet a été mené selon la méthodologie **Scrum**, organisée en deux releases et cinq sprints.

**Mots clés :** Véhicule Électrique, Borne de Recharge, Supervision Temps Réel, Go, GraphQL, React, Scrum

# Abstract

This report summarizes the end-of-studies project for the computer engineering diploma at Tek-Up University, completed at EntityService.

The objective is the design and development of an integrated platform for real-time management and supervision of electric vehicle charging stations. The platform consists of three components: an automatic data collector, a mobile app for field technicians, and a web supervision dashboard. The system is built on a modern architecture combining a **Go** backend with **GraphQL** API, a **React** dashboard, a **React Native** mobile app, and an observability stack (**ELK**, **Prometheus**, **Grafana**). The project followed **Scrum** agile methodology, organized into two releases and five sprints.

**Keywords :** Electric Vehicle, Charging Station, Real-Time Supervision, Go, GraphQL, React, React Native, Scrum

---